

Fantasy Football Project: Complete Code Walkthrough in Execution Order

Generated on: 2026-02-15

This document explains all source code in this repository, including modules, classes, methods, wrappers, and notebook drivers. The order follows actual notebook execution first, then deep dives by file.

1) Primary Execution Order (Notebook-First)

The main runtime entry point is `Fantasy_Football_Analysis.ipynb`. In normal use, users run notebook cells top-to-bottom. The sections below list code cells in execution order and explain what each stage does.

Cell 6 (EXECUTABLE)

- First executable line: `import subprocess`
- Source lines: 32
- Direct calls: `install_package`, `print`, `subprocess.check_call`

Cell 8 (EXECUTABLE)

- First executable line: `import os`
- Source lines: 9
- Direct calls: none (or commented-only cell)

Cell 11 (COMMENTED_REFERENCE)

- First executable line: (commented / reference)
- Source lines: 26
- Direct calls: none (or commented-only cell)

Cell 12 (COMMENTED_REFERENCE)

- First executable line: (commented / reference)
- Source lines: 30
- Direct calls: none (or commented-only cell)

Cell 14 (EXECUTABLE)

- First executable line: `analyzer = DraftValueAnalyzer(`
- Source lines: 19
- Direct calls: `DraftValueAnalyzer`, `analyzer.run_full_pipeline`, `len`, `print`, `range`
- Runtime role: starts the full draft pipeline (`DraftValueAnalyzer.run_full_pipeline`) and populates shared analyzer state used by later stages.

Cell 16 (EXECUTABLE)

- First executable line: `lineups_filt = analyzer.lineups_filt`
- Source lines: 42
- Direct calls: `ValueError`, `analyzer.compute_expected_values`, `analyzer.score_picks`, `draft_scored.to_csv`, `head`, `len`, `print`
- Runtime role: computes expected-value baselines and scores each draft pick against year, pooled, and polynomial baselines.

Cell 18 (EXECUTABLE)

- First executable line: `team_totals_auto = analyzer.plot_team_total_valid_points_distribution()`
- Source lines: 14
- Direct calls: `analyzer.plot_team_total_valid_points_distribution`

Cell 20 (EXECUTABLE)

- First executable line: `analyzer.plot_expected_by_pick()`
- Source lines: 7
- Direct calls: `analyzer.plot_expected_by_pick`

Cell 22 (EXECUTABLE)

- First executable line: `analyzer.plot_per_season_expected_values(zoom_first_25=True)`
- Source lines: 3
- Direct calls: `analyzer.plot_per_season_expected_values`

Cell 24 (EXECUTABLE)

- First executable line: `advantage_by_year = analyzer.plot_human_advantage_by_year(baseline="Poly")`
- Source lines: 17
- Direct calls: `advantage_by_round.head`, `analyzer.plot_human_advantage_by_round`, `analyzer.plot_human_advantage_by_round`, `print`

Cell 26 (EXECUTABLE)

- First executable line: `transactions_raw = analyzer.load_multi_season_transactions(lineups_filt=lineups_filt)`
- Source lines: 5
- Direct calls: `analyzer.load_multi_season_transactions`, `len`, `print`, `to_dict`, `value_counts`
- Runtime role: waiver pipeline stages (load transactions -> build stints -> compute valid stint points -> compute baseline candidates).

Cell 28 (EXECUTABLE)

- First executable line: `waiver_stints = analyzer.build_waiver_add_stints()`
- Source lines: 12
- Direct calls: `analyzer.build_waiver_add_stints`, `len`, `print`, `waiver_stints.head`
- Runtime role: waiver pipeline stages (load transactions -> build stints -> compute valid stint points -> compute baseline candidates).

Cell 30 (EXECUTABLE)

- First executable line: `waiver_with_valid = analyzer.compute_valid_waiver_points()`
- Source lines: 9
- Direct calls: `analyzer.compute_valid_waiver_points`, `head`, `len`, `print`, `waiver_with_valid.sort_values`
- Runtime role: waiver pipeline stages (load transactions -> build stints -> compute valid stint points -> compute baseline candidates).

Cell 32 (EXECUTABLE)

- First executable line: `baseline_candidates, team_season = analyzer.compute_waiver_baseline_candidates(waiver_with_valid)`
- Source lines: 14
- Direct calls: `analyzer.compute_waiver_baseline_candidates`, `baseline_candidates.keys`, `baseline_candidates.values`, `list`, `pd.DataFrame`, `print`, `sort_values`
- Runtime role: waiver pipeline stages (load transactions -> build stints -> compute valid stint points -> compute baseline candidates).

Cell 34 (EXECUTABLE)

- First executable line: `from src.analysis.notebook_workflow import NotebookWorkflow`
- Source lines: 22
- Direct calls: `NotebookWorkflow`, `globals`, `print`, `range`, `wf.block_draft_pipeline`, `wf.block_waiver_pipeline`, `wf.plot_cumulative_draft_waiver`, `wf.plot_waiver_baseline_exploration`, `wf.plot_yearly_draft_waiver_tot`

Cell 36 (EXECUTABLE)

- First executable line: `startsit_weekly, startsit_weekly_clean = analyzer.compute_startsit_metrics()`
- Source lines: 13
- Direct calls: `analyzer.compute_startsit_metrics`, `describe`, `len`, `print`
- Runtime role: computes start/sit quality metrics against projected-optimal lineups.

Cell 38 (EXECUTABLE)

- First executable line: `startsit_summary_wf = wf.plot_startsit_by_year()`
- Source lines: 6
- Direct calls: `print`, `wf.block_trade_stats`, `wf.plot_startsit_by_year`

Cell 42 (EXECUTABLE)

- First executable line: `from src.analysis.notebook_workflow import NotebookWorkflow`
- Source lines: 1
- Direct calls: none (or commented-only cell)

Cell 43 (EXECUTABLE)

- First executable line: `wf = NotebookWorkflow()`
- Source lines: 14
- Direct calls: `NotebookWorkflow`, `print`, `range`, `wf.block_draft_pipeline`, `wf.block_startsit`, `wf.block_waiver_pipeline`
- Runtime role: wrapper-based one-line orchestration via `NotebookWorkflow` instead of manual staged calls.

Cell 44 (EXECUTABLE)

- First executable line: `draft_scored = wf.state.draft_scored`
- Source lines: 16
- Direct calls: `float`, `print`

Cell 45 (EXECUTABLE)

- First executable line: `import numpy as np`
- Source lines: 116
- Direct calls: `active_stints_week.merge`, `astype`, `ax1.axhline`, `ax1.bar`, `ax1.get_legend_handles_labels`, `ax1.set_xlabel`, `ax1.set_xticklabels`, `ax1.set_xticks`, `ax1.set_ylabel`, `ax1.set_ylim`, `ax1.set_zorder`, `ax1.twinx`

Cell 48 (EXECUTABLE)

- First executable line: `injury_results = collect_injury_data()`
- Source lines: 8
- Direct calls: `collect_injury_data`, `print`, `range`
- Runtime role: injury data pull + injury analysis pipeline from raw load through aggregate output save.

Cell 49 (EXECUTABLE)

- First executable line: `injury_analyzer = InjuryAnalyzer()`
- Source lines: 7
- Direct calls: `InjuryAnalyzer`, `range`
- Runtime role: injury data pull + injury analysis pipeline from raw load through aggregate output save.

Cell 50 (EXECUTABLE)

- First executable line: `injury_analyzer.load_injury_data()`
- Source lines: 3
- Direct calls: `injury_analyzer.load_injury_data`, `injury_analyzer.load_lineup_data`
- Runtime role: injury data pull + injury analysis pipeline from raw load through aggregate output save.

Cell 51 (EXECUTABLE)

- First executable line: `injury_analyzer.calculate_player_baselines()`
- Source lines: 2
- Direct calls: `injury_analyzer.calculate_player_baselines`
- Runtime role: injury data pull + injury analysis pipeline from raw load through aggregate output save.

Cell 52 (EXECUTABLE)

- First executable line: `injury_analyzer.merge_injury_lineup_data()`
- Source lines: 2
- Direct calls: `injury_analyzer.merge_injury_lineup_data`
- Runtime role: injury data pull + injury analysis pipeline from raw load through aggregate output save.

Cell 53 (EXECUTABLE)

- First executable line: `analysis_df = injury_analyzer.prepare_analysis_data()`
- Source lines: 2
- Direct calls: `injury_analyzer.prepare_analysis_data`
- Runtime role: injury data pull + injury analysis pipeline from raw load through aggregate output save.

Cell 54 (EXECUTABLE)

- First executable line: `agg_stats = injury_analyzer.compute_aggregated_stats(analysis_df)`
- Source lines: 5
- Direct calls: `injury_analyzer.compute_aggregated_stats`
- Runtime role: injury data pull + injury analysis pipeline from raw load through aggregate output save.

Cell 55 (EXECUTABLE)

- First executable line: `output_dir = Path("data/preprocessed")`
- Source lines: 5
- Direct calls: `Path`, `agg_stats.to_csv`, `individual_records.to_csv`, `output_dir.mkdir`
- Runtime role: injury data pull + injury analysis pipeline from raw load through aggregate output save.

Cell 58 (EXECUTABLE)

- First executable line: `injury_analyzer.plot_heatmap_type_position()`
- Source lines: 4
- Direct calls: `injury_analyzer.plot_heatmap_type_position`, `plt.show`

Cell 60 (EXECUTABLE)

- First executable line: `injury_analyzer.plot_heatmap_type_status()`
- Source lines: 4
- Direct calls: `injury_analyzer.plot_heatmap_type_status`, `plt.show`

Cell 62 (EXECUTABLE)

- First executable line: `injury_analyzer.plot_heatmap_position_status()`
- Source lines: 4
- Direct calls: `injury_analyzer.plot_heatmap_position_status`, `plt.show`

Cell 65 (EXECUTABLE)

- First executable line: `injury_analyzer.plot_point_differential_distribution()`
- Source lines: 5
- Direct calls: `injury_analyzer.plot_point_differential_distribution`, `plt.show`

Cell 67 (EXECUTABLE)

- First executable line: `print("Top 10 Injury Impact Combinations (by average point differential):")`
- Source lines: 6
- Direct calls: `agg_stats.nlargest`, `agg_stats.nsmallest`, `print`

Cell 70 (COMMENTED_REFERENCE)

- First executable line: (commented / reference)
- Source lines: 60
- Direct calls: none (or commented-only cell)
- Runtime role: archived prototype/reference code; not used in current active pipeline.

Cell 72 (COMMENTED_REFERENCE)

- First executable line: (commented / reference)
- Source lines: 80
- Direct calls: none (or commented-only cell)
- Runtime role: archived prototype/reference code; not used in current active pipeline.

2) High-Level Runtime Call Chains

Draft baseline chain (active path)

1. Notebook cell 14 creates `DraftValueAnalyzer` and calls `run_full_pipeline`.
2. `run_full_pipeline` internally calls: `clean_raw_data -> load_multi_season_data -> _add_draft_length -> filter_standard_leagues -> filter_scoring_rule_outliers -> filter_draft_length -> enrich_draft_data -> compute_optimal_startable_points -> add_valid_points`.
3. Notebook cell 16 calls `compute_expected_values` and `score_picks` to derive points-added metrics.
4. Visualization methods in cells 18/20/22/24 consume these outputs.

Waiver chain (active path)

1. `load_multi_season_transactions`
2. `build_waiver_add_stints`
3. `compute_valid_waiver_points`
4. `compute_waiver_baseline_candidates`
5. Plot wrappers (`plot_waiver_baseline_exploration`, `plot_draft_vs_waiver_points`, `plot_yearly_draft_waiver`)

Start/sit chain (active path)

1. `compute_startsit_metrics` (projected-optimal baseline)
2. `plot_startsit_by_year` and `plot_cumulative_draft_waiver_startsit`

Injury chain (active path)

1. `collect_injury_data` for season files
2. `InjuryAnalyzer` setup
3. `load_injury_data` + `load_lineup_data`
4. `calculate_player_baselines`
5. `merge_injury_lineup_data`
6. `prepare_analysis_data` + `compute_aggregated_stats` (or `run_full_analysis`)
7. Heatmaps/distribution plots and CSV output

3) File-by-File Deep Dive (All Python Code)

File: `src\__init__.py`

Module role: package init or utility module with no top-level docstring.

Top-level content summary: - # Package initialization

File: `src\data_fetchers\__init__.py`

Module role: package init or utility module with no top-level docstring.

Top-level content summary: - # Data fetchers package - from `.data_collector` import `FantasyDataCollector`
- `__all__ = ['FantasyDataCollector']`

File: `src\data_fetchers\espn_random_public_league_fetcher_seeded_final_pull.py`

Module role: Harvest EXACTLY N ESPN fantasy leagues that match: - 10 teams - 1.0 PPR (receptions `statId == 53` -> `points == 1.0`)

`_ensure_dir_for_file` (lines 63-72)

- Kind: function
- Signature: `_ensure_dir_for_file(path)` -> `None`
- Intent: Ensure the directory for a file path exists, creating it if necessary.
- Validation/Error handling: no explicit raises in this callable; errors are mostly delegated to called APIs.
- Core data operations: mostly control flow/IO logic, not heavy `DataFrame` transforms.
- Calls out to: `dirname`, `exists`, `os.makedirs`
- Called by (static reference): `write_csv_rows`

`read_csv_rows` (lines 79-94)

- Kind: function
- Signature: `read_csv_rows(path)` -> `List[Dict[str, Any]]`
- Intent: Read all rows from a CSV file into a list of dictionaries.
- Validation/Error handling: no explicit raises in this callable; errors are mostly delegated to called APIs.
- Core data operations: `read_csv` (Loads persisted raw/intermediate CSV inputs.)
- Calls out to: `csv.DictReader`, `dict`, `exists`, `open`
- Called by (static reference): `harvest_target_leagues`

write_csv_rows (lines 97-133)

- Kind: function
- Signature: `write_csv_rows(path, rows, dedupe_cols) -> None`
- Intent: Write rows to CSV file, removing duplicates based on specified columns.
- Validation/Error handling: no explicit raises in this callable; errors are mostly delegated to called APIs.
- Core data operations: mostly control flow/IO logic, not heavy DataFrame transforms.
- Calls out to: `_ensure_dir_for_file`, `cols_set.update`, `csv.DictWriter`, `list`, `open`, `out.append`, `r.get`, `r.keys`, `seen.add`, `set`, `str`, `tuple`, `w.writeheader`, `w.writerows`
- Called by (static reference): `harvest_target_leagues`

fetch_league_json (lines 144-182)

- Kind: function
- Signature: `fetch_league_json(league_id, season, timeout, session) -> Tuple[Optional[Dict[str, Any]], int]`
- Intent: Fetch league JSON data from ESPN API.
- Validation/Error handling: no explicit raises in this callable; errors are mostly delegated to called APIs.
- Core data operations: mostly control flow/IO logic, not heavy DataFrame transforms.
- Side effects: HTTP API access
- Calls out to: `int`, `r.json`, `session.get`
- Called by (static reference): `fetch_with_retries`

fetch_with_retries (lines 185-226)

- Kind: function
- Signature: `fetch_with_retries(league_id, season, timeout, session) -> Tuple[Optional[Dict[str, Any]], int]`
- Intent: Fetch league JSON with automatic retries for transient errors.
- Validation/Error handling: no explicit raises in this callable; errors are mostly delegated to called APIs.
- Core data operations: mostly control flow/IO logic, not heavy DataFrame transforms.
- Side effects: HTTP API access
- Calls out to: `fetch_league_json`, `random.random`, `range`, `time.sleep`
- Called by (static reference): `harvest_target_leagues`

extract_meta (lines 233-266)

- Kind: function
- Signature: `extract_meta(league_json, league_id, season) -> Dict[str, Any]`
- Intent: Extract key metadata from league JSON response.
- Validation/Error handling: no explicit raises in this callable; errors are mostly delegated to called APIs.
- Core data operations: mostly control flow/IO logic, not heavy DataFrame transforms.
- Calls out to: `int`, `it.get`, `league_json.get`, `scoring.get`, `settings.get`
- Called by (static reference): `harvest_target_leagues`

is_target (lines 272-293)

- Kind: function
- Signature: `is_target(meta) -> bool`
- Intent: Check if league matches target criteria: 10 teams and 1.0 PPR.
- Validation/Error handling: no explicit raises in this callable; errors are mostly delegated to called APIs.

- Core data operations: mostly control flow/IO logic, not heavy DataFrame transforms.
- Calls out to: `abs`, `float`, `int`, `meta.get`
- Called by (static reference): `harvest_target_leagues`

choose_id (lines 313-335)

- Kind: function
- Signature: `choose_id(seed_ids, rng) -> int`
- Intent: Choose next league ID to probe using seeded sampling strategy.
- Validation/Error handling: no explicit raises in this callable; errors are mostly delegated to called APIs.
- Core data operations: mostly control flow/IO logic, not heavy DataFrame transforms.
- Calls out to: `max`, `rng.choice`, `rng.randint`, `rng.random`
- Called by (static reference): `harvest_target_leagues`

adapt_delay (lines 338-366)

- Kind: function
- Signature: `adapt_delay(state, recent) -> None`
- Intent: Adaptively adjust request delay based on recent error patterns.
- Validation/Error handling: no explicit raises in this callable; errors are mostly delegated to called APIs.
- Core data operations: mostly control flow/IO logic, not heavy DataFrame transforms.
- Calls out to: `Counter`, `c.get`, `len`, `max`, `min`, `sum`
- Called by (static reference): `harvest_target_leagues`

harvest_target_leagues (lines 373-546)

- Kind: function
- Signature: `harvest_target_leagues(target_n, season, seeds_csv, out_targets_csv, exists_401_csv, timeout, rng_seed) -> List[Dict[str, Any]]`
- Intent: Harvest target leagues from ESPN API matching criteria (10 teams, 1.0 PPR).
- Validation/Error handling: contains 1 explicit `raise` statements or guard failures.
- Core data operations: `read_csv` (Loads persisted raw/intermediate CSV inputs.)
- Side effects: HTTP API access
- Calls out to: `Counter`, `RuntimeError`, `State`, `adapt_delay`, `choose_id`, `deque`, `exists_401.append`, `exists_401_set.add`, `extract_meta`, `fetch_with_retries`, `float`, `int`, `is_target`, `join`, `len`
- Called by (static reference): notebook entrypoint or external caller, or no internal references detected.
- Detailed execution notes:
 - Large-scale public league discovery routine. Uses seeded neighborhood sampling with occasional global exploration and adaptive request delay based on recent HTTP statuses.
 - Persists both targets and 401-private IDs for resume safety and reduced duplicate probing.

File: `src\data_fetchers\data_collector.py`

Module role: package init or utility module with no top-level docstring.

FantasyDataCollector.__init__ (lines 12-25)

- Kind: method
- Signature: `FantasyDataCollector.__init__(self, swid, espn_s2, verbose) -> None`
- Intent: Initialize data collector. Credentials optional for public leagues.
- Validation/Error handling: no explicit raises in this callable; errors are mostly delegated to called APIs.
- Core data operations: mostly control flow/IO logic, not heavy DataFrame transforms.

- Calls out to: `FantasyDataCollector._ensure_dependencies`, `Lock`, `abspath`, `dirname`, `join`, `normpath`, `os.makedirs`
- Called by (static reference): notebook entrypoint or external caller, or no internal references detected.

`FantasyDataCollector._ensure_dependencies` (lines 27-38)

- Kind: method
- Signature: `FantasyDataCollector._ensure_dependencies(self) -> None`
- Intent: Install dependencies if they are not already installed.
- Validation/Error handling: no explicit raises in this callable; errors are mostly delegated to called APIs.
- Core data operations: mostly control flow/IO logic, not heavy `DataFrame` transforms.
- Calls out to: `importlib.import_module`, `importlib.invalidate_caches`, `print`, `subprocess.check_call`
- Called by (static reference): `FantasyDataCollector.__init__`

`FantasyDataCollector._get_csv_row_count` (lines 40-46)

- Kind: method
- Signature: `FantasyDataCollector._get_csv_row_count(self, filename) -> None`
- Intent: Count rows in CSV file (excluding header) without loading into memory.
- Validation/Error handling: no explicit raises in this callable; errors are mostly delegated to called APIs.
- Core data operations: mostly control flow/IO logic, not heavy `DataFrame` transforms.
- Calls out to: `exists`, `join`, `max`, `open`, `sum`
- Called by (static reference): notebook entrypoint or external caller, or no internal references detected.

`FantasyDataCollector._save_to_csv` (lines 48-63)

- Kind: method
- Signature: `FantasyDataCollector._save_to_csv(self, data, filename) -> None`
- Intent: Append data to CSV file in thread-safe manner.
- Validation/Error handling: no explicit raises in this callable; errors are mostly delegated to called APIs.
- Core data operations: `to_csv` (Writes intermediate or final outputs for reuse by later notebook stages.)
- Side effects: CSV write side effect
- Calls out to: `df.to_csv`, `getsize`, `isfile`, `join`, `pd.DataFrame`
- Called by (static reference): `FantasyDataCollector._get_seasonal_data`, `FantasyDataCollector.extract_league`

`FantasyDataCollector._print_object_info` (lines 65-104)

- Kind: method
- Signature: `FantasyDataCollector._print_object_info(self, obj, obj_name) -> None`
- Intent: Print all public methods and attributes of an object (debugging helper).
- Validation/Error handling: no explicit raises in this callable; errors are mostly delegated to called APIs.
- Core data operations: mostly control flow/IO logic, not heavy `DataFrame` transforms.
- Calls out to: `attr.startswith`, `attributes.append`, `callable`, `dir`, `getattr`, `len`, `methods.append`, `print`, `sorted`, `str`, `type`
- Called by (static reference): `FantasyDataCollector._get_draft_data`, `FantasyDataCollector._get_seasonal_data`, `FantasyDataCollector.extract_league`

`FantasyDataCollector._league_already_processed` (lines 106-116)

- Kind: method
- Signature: `FantasyDataCollector._league_already_processed(self, league_id, year) -> None`

- Intent: Check if league/year combination exists in draft_data.csv.
- Validation/Error handling: no explicit raises in this callable; errors are mostly delegated to called APIs.
- Core data operations: read_csv (Loads persisted raw/intermediate CSV inputs.)
- Calls out to: any, exists, join, pd.read_csv
- Called by (static reference): FantasyDataCollector.extract_league

FantasyDataCollector.extract_league (lines 118-148)

- Kind: method
- Signature: FantasyDataCollector.extract_league(self, league_id, year, skip_existing) -> None
- Intent: Main entry point for processing a league.
- Validation/Error handling: no explicit raises in this callable; errors are mostly delegated to called APIs.
- Core data operations: to_csv (Writes intermediate or final outputs for reuse by later notebook stages.)
- Calls out to: FantasyDataCollector._get_draft_data, FantasyDataCollector._get_seasonal_data, FantasyDataCollector._league_already_processed, FantasyDataCollector._print_object_info, FantasyDataCollector._save_to_csv, League, len, print
- Called by (static reference): notebook entrypoint or external caller, or no internal references detected.
- Detailed execution notes:
 - Main league-year extraction entry point. It instantiates espn_api League, pulls draft detail from raw REST for autoDraftTypeId, then derives weekly lineups and inferred transactions.
 - Skip-existing logic prevents duplicate processing for already harvested league-year records.

FantasyDataCollector._get_draft_data (lines 150-209)

- Kind: method
- Signature: FantasyDataCollector._get_draft_data(self, league, lid, year) -> None
- Intent: Fetch draft data via raw ESPN API to get autoDraftTypeId.
- Validation/Error handling: no explicit raises in this callable; errors are mostly delegated to called APIs.
- Core data operations: mostly control flow/IO logic, not heavy DataFrame transforms.
- Side effects: HTTP API access
- Calls out to: FantasyDataCollector._print_object_info, data.get, get, p.get, player_map.get, print, r.json, r.raise_for_status, requests.get, rows.append
- Called by (static reference): FantasyDataCollector.extract_league
- Detailed execution notes:
 - Calls ESPN mDraftDetail endpoint directly because espn_api objects do not expose autoDraftTypeId. This identifies manual vs autodrafted picks at pick level.

FantasyDataCollector._get_seasonal_data (lines 211-335)

- Kind: method
- Signature: FantasyDataCollector._get_seasonal_data(self, league, lid, year, batch_writes) -> None
- Intent: Processes weekly data and identifies reciprocal trades.
- Validation/Error handling: no explicit raises in this callable; errors are mostly delegated to called APIs.
- Core data operations: to_csv (Writes intermediate or final outputs for reuse by later notebook stages.)
- Calls out to: FantasyDataCollector._print_object_info, FantasyDataCollector._save_to_csv, all_lineups.extend, all_transactions.extend, any, arrivals.items, curr_rosters.items, curr_rosters.values, curr_week_lineups.append, curr_week_trans.append, departures.get, departures.items, getattr, league.box_scores, master_rosters_prev.get
- Called by (static reference): FantasyDataCollector.extract_league

- Detailed execution notes:
 - Builds weekly lineup rows from box scores and infers transaction actions by comparing team rosters week-over-week.
 - Trade inference is heuristic: arriving player was previously rostered plus reciprocal movement indicates trade-like behavior; otherwise treated as waiver add.

`collect_leagues_parallel` (lines 338-491)

- Kind: function
- Signature: `collect_leagues_parallel(league_ids, years, max_workers, skip_existing, swid, espn_s2, verbose, data_dir) -> None`
- Intent: High-level function to collect data from multiple leagues and years in parallel.
- Validation/Error handling: contains 4 explicit `raise` statements or guard failures.
- Core data operations: `astype`; `read_csv` (Loads persisted raw/intermediate CSV inputs.)
- Calls out to: `FantasyDataCollector`, `FileNotFoundError`, `ThreadPoolExecutor`, `TypeError`, `ValueError`, `as_completed`, `astype`, `collector.extract_league`, `executor.submit`, `exists`, `future.result`, `hasattr`, `isinstance`, `join`, `len`
- Called by (static reference): notebook entrypoint or external caller, or no internal references detected.
- Detailed execution notes:
 - Parallel harvesting wrapper over `FantasyDataCollector.extract_league`. Normalizes ID/year inputs, creates per-year output folders, and tracks success/failure summaries.

`collect_injury_data` (lines 494-579)

- Kind: function
- Signature: `collect_injury_data(years, output_dir) -> None`
- Intent: Collect NFL injury data for specified years using `nfl_data_py`.
- Validation/Error handling: contains 1 explicit `raise` statements or guard failures.
- Core data operations: `to_csv` (Writes intermediate or final outputs for reuse by later notebook stages.)
- Side effects: CSV write side effect
- Calls out to: `TypeError`, `abspath`, `dirname`, `files_saved.append`, `importlib.import_module`, `importlib.invalidate_caches`, `injuries_df.to_csv`, `isinstance`, `join`, `len`, `list`, `nfl.import_injuries`, `normpath`, `os.makedirs`, `print`
- Called by (static reference): notebook entrypoint or external caller, or no internal references detected.
- Detailed execution notes:
 - Uses `nfl_data_py import_injuries` for each requested season and saves season-specific CSVs under `data/raw/injuries`.

File: `src\analysis\__init__.py`

Module role: Analysis package exports.

Top-level content summary: - `"""Analysis package exports."""` - - ``from .draft_value_analyzer import DraftValueAnalyzer`` - ``from .injury_analyzer import InjuryAnalyzer`` - ``from .notebook_workflow import NotebookWorkflow, WorkflowState`` - - `__all__ = [- "DraftValueAnalyzer", - "InjuryAnalyzer", - "NotebookWorkflow", - "WorkflowState", - ]`

File: `src\analysis\draft_value_analyzer.py`

Module role: Draft Value Analysis Module

`DraftValueAnalyzer.__init__` (lines 31-90)

- Kind: method
- Signature: `DraftValueAnalyzer.__init__(self, raw_base, out_dir, years, expected_starters, expected_draft_length, anchors_by_pos, anchor_synonyms, verbose) -> None`

- Intent: Initialize the Draft Value Analyzer.
- Validation/Error handling: no explicit raises in this callable; errors are mostly delegated to called APIs.
- Core data operations: mostly control flow/IO logic, not heavy DataFrame transforms.
- Calls out to: `Path`, `makedirs`, `range`
- Called by (static reference): notebook entrypoint or external caller, or no internal references detected.

DraftValueAnalyzer.normalize_player_name (lines 99-114)

- Kind: method
- Signature: `DraftValueAnalyzer.normalize_player_name(name, synonyms) -> str`
- Intent: Normalize player name for consistent matching.
- Validation/Error handling: no explicit raises in this callable; errors are mostly delegated to called APIs.
- Core data operations: mostly control flow/IO logic, not heavy DataFrame transforms.
- Calls out to: `pd.isna`, `re.sub`, `replace`, `s.replace`, `str`, `strip`, `synonyms.get`
- Called by (static reference): `DraftValueAnalyzer._load_draft`, `DraftValueAnalyzer._load_lineups`, `DraftValueAnalyzer.enrich_draft_data`, `DraftValueAnalyzer.filter_scoring_rule_outliers`, `DraftValueAnalyzer.load_transactions`, `InjuryAnalyzer.load_injury_data`, `InjuryAnalyzer.load_lineup_data`

DraftValueAnalyzer.normalize_slot (lines 119-137)

- Kind: method
- Signature: `DraftValueAnalyzer.normalize_slot(slot) -> str`
- Intent: Normalize lineup slot designation.
- Validation/Error handling: no explicit raises in this callable; errors are mostly delegated to called APIs.
- Core data operations: mostly control flow/IO logic, not heavy DataFrame transforms.
- Calls out to: `str`, `strip`, `upper`
- Called by (static reference): notebook entrypoint or external caller, or no internal references detected.

DraftValueAnalyzer.clean_raw_data (lines 143-172)

- Kind: method
- Signature: `DraftValueAnalyzer.clean_raw_data(self) -> None`
- Intent: Drop duplicate rows in raw CSV files.
- Validation/Error handling: no explicit raises in this callable; errors are mostly delegated to called APIs.
- Core data operations: `drop_duplicates`; `to_csv` (Writes intermediate or final outputs for reuse by later notebook stages.); `read_csv` (Loads persisted raw/intermediate CSV inputs.)
- Side effects: CSV write side effect
- Calls out to: `df.drop_duplicates`, `df2.to_csv`, `len`, `path.exists`, `pd.read_csv`, `print`, `str`
- Called by (static reference): `DraftValueAnalyzer.run_full_pipeline`, `NotebookWorkflow.block_clean_raw`

DraftValueAnalyzer.load_multi_season_data (lines 174-222)

- Kind: method
- Signature: `DraftValueAnalyzer.load_multi_season_data(self) -> Tuple[pd.DataFrame, pd.DataFrame]`
- Intent: Load draft and lineup data across multiple seasons.
- Validation/Error handling: contains 2 explicit `raise` statements or guard failures.
- Core data operations: `concat`
- Calls out to: `DraftValueAnalyzer._load_draft`, `DraftValueAnalyzer._load_lineups`, `FileNotFoundError`, `dpath.exists`, `draft_parts.append`, `int`, `isdigit`, `iterdir`, `len`, `lineup_parts.append`, `lpath.exists`, `p.is_dir`, `pd.concat`, `print`, `sorted`

- Called by (static reference): `DraftValueAnalyzer.run_full_pipeline`, `NotebookWorkflow.block_load_data`

`DraftValueAnalyzer._load_draft` (lines 224-242)

- Kind: method
- Signature: `DraftValueAnalyzer._load_draft(self, path, year) -> pd.DataFrame`
- Intent: Load and normalize draft data for a single year.
- Validation/Error handling: contains 1 explicit `raise` statements or guard failures.
- Core data operations: `fillna`; `astype`; `to_numeric`; `read_csv` (Loads persisted raw/intermediate CSV inputs.)
- Calls out to: `DraftValueAnalyzer.normalize_player_name`, `ValueError`, `astype`, `df.copy`, `fillna`, `int`, `map`, `pd.read_csv`, `pd.to_numeric`
- Called by (static reference): `DraftValueAnalyzer.load_multi_season_data`

`DraftValueAnalyzer._load_lineups` (lines 244-264)

- Kind: method
- Signature: `DraftValueAnalyzer._load_lineups(self, path, year) -> pd.DataFrame`
- Intent: Load and normalize lineup data for a single year.
- Validation/Error handling: contains 1 explicit `raise` statements or guard failures.
- Core data operations: `astype`; `to_numeric`; `read_csv` (Loads persisted raw/intermediate CSV inputs.)
- Calls out to: `DraftValueAnalyzer.normalize_player_name`, `ValueError`, `astype`, `df.copy`, `int`, `map`, `pd.read_csv`, `pd.to_numeric`, `strip`
- Called by (static reference): `DraftValueAnalyzer.load_multi_season_data`

`DraftValueAnalyzer.filter_standard_leagues` (lines 268-309)

- Kind: method
- Signature: `DraftValueAnalyzer.filter_standard_leagues(self, draft, lineups) -> Tuple[pd.DataFrame, pd.DataFrame, pd.DataFrame]`
- Intent: Filter to leagues with standard starter configurations.
- Validation/Error handling: no explicit raises in this callable; errors are mostly delegated to called APIs.
- Core data operations: `merge` (Joins datasets to align draft, lineup, transaction, and baseline signals by shared keys.); `value_counts`
- Calls out to: `DraftValueAnalyzer._infer_league_starter_signature`, `draft.merge`, `head`, `int`, `len`, `lineups.merge`, `print`, `sum`, `value_counts`
- Called by (static reference): `DraftValueAnalyzer.run_full_pipeline`, `NotebookWorkflow.block_filter_leagues`

`DraftValueAnalyzer._infer_league_starter_signature` (lines 311-390)

- Kind: method
- Signature: `DraftValueAnalyzer._infer_league_starter_signature(self, lineups) -> pd.DataFrame`
- Intent: Infer starter configuration for each league-year.
- Validation/Error handling: contains 1 explicit `raise` statements or guard failures.
- Core data operations: `groupby` (Performs grouped aggregations to convert event-level rows into season or week summaries.); `pivot` (Reshapes grouped results into matrix form for signature inference or heatmaps.); `pivot_table` (Reshapes grouped results into matrix form for signature inference or heatmaps.); `agg` (Computes multi-metric summaries (mean/count/std/quantiles) used in baseline construction.); `fillna`; `astype`; `value_counts`; `reset_index`
- Calls out to: `ValueError`, `agg`, `astype`, `copy`, `fillna`, `int`, `len`, `lineups.copy`, `map`, `pivot.groupby`, `reset_index`, `s.value_counts`, `set`, `size`, `sorted`
- Called by (static reference): `DraftValueAnalyzer.filter_standard_leagues`

DraftValueAnalyzer.filter_scoring_rule_outliers (lines 392-462)

- Kind: method
- Signature: `DraftValueAnalyzer.filter_scoring_rule_outliers(self, draft, lineups, z_thresh, min_anchors_hit) -> Tuple[pd.DataFrame, pd.DataFrame, pd.DataFrame]`
- Intent: Filter out league-years with unusual scoring rules using anchor players.
- Validation/Error handling: no explicit raises in this callable; errors are mostly delegated to called APIs.
- Core data operations: `groupby` (Performs grouped aggregations to convert event-level rows into season or week summaries.); `merge` (Joins datasets to align draft, lineup, transaction, and baseline signals by shared keys.); `agg` (Computes multi-metric summaries (mean/count/std/quantiles) used in baseline construction.); `transform`; `drop_duplicates`; `astype`; `sort_values`; `reset_index`
- Calls out to: `DraftValueAnalyzer._season_points_by_league_year`, `DraftValueAnalyzer.normalize_player_name`, `a.groupby`, `abs`, `agg`, `anchor_rows.append`, `draft.merge`, `drop_duplicates`, `float`, `head`, `items`, `len`, `lineups.merge`, `np.abs`, `np.isnan`
- Called by (static reference): `DraftValueAnalyzer.run_full_pipeline`, `NotebookWorkflow.block_filter_leagues`

DraftValueAnalyzer._season_points_by_league_year (lines 464-473)

- Kind: method
- Signature: `DraftValueAnalyzer._season_points_by_league_year(self, lineups) -> pd.DataFrame`
- Intent: Compute season total points by league-year-player.
- Validation/Error handling: no explicit raises in this callable; errors are mostly delegated to called APIs.
- Core data operations: `groupby` (Performs grouped aggregations to convert event-level rows into season or week summaries.); `to_numeric`; `reset_index`
- Calls out to: `df.groupby`, `lineups.copy`, `pd.to_numeric`, `rename`, `reset_index`, `sum`
- Called by (static reference): `DraftValueAnalyzer.filter_scoring_rule_outliers`

DraftValueAnalyzer.filter_draft_length (lines 475-503)

- Kind: method
- Signature: `DraftValueAnalyzer.filter_draft_length(self, draft, lineups) -> Tuple[pd.DataFrame, pd.DataFrame]`
- Intent: Filter to leagues with expected draft length.
- Validation/Error handling: no explicit raises in this callable; errors are mostly delegated to called APIs.
- Core data operations: `groupby` (Performs grouped aggregations to convert event-level rows into season or week summaries.); `merge` (Joins datasets to align draft, lineup, transaction, and baseline signals by shared keys.); `reset_index`
- Calls out to: `DraftValueAnalyzer._add_draft_length`, `d.groupby`, `d.merge`, `draft.copy`, `int`, `lineups.merge`, `max`, `print`, `reset_index`
- Called by (static reference): `DraftValueAnalyzer.run_full_pipeline`, `NotebookWorkflow.block_filter_leagues`

DraftValueAnalyzer._add_draft_length (lines 505-513)

- Kind: method
- Signature: `DraftValueAnalyzer._add_draft_length(self, draft_df) -> pd.DataFrame`
- Intent: Add draft length column.
- Validation/Error handling: no explicit raises in this callable; errors are mostly delegated to called APIs.
- Core data operations: `groupby` (Performs grouped aggregations to convert event-level rows into season or week summaries.); `transform`; `fillna`; `astype`; `to_numeric`
- Calls out to: `astype`, `draft_df.copy`, `fillna`, `out.groupby`, `pd.to_numeric`, `transform`

- Called by (static reference): `DraftValueAnalyzer.filter_draft_length`, `DraftValueAnalyzer.run_full_pipeline`, `NotebookWorkflow.block_filter_leagues`

`DraftValueAnalyzer.enrich_draft_data` (lines 517-569)

- Kind: method
- Signature: `DraftValueAnalyzer.enrich_draft_data(self, draft, lineups) -> pd.DataFrame`
- Intent: Enrich draft data with position and season total points.
- Validation/Error handling: no explicit raises in this callable; errors are mostly delegated to called APIs.
- Core data operations: `merge` (Joins datasets to align draft, lineup, transaction, and baseline signals by shared keys.); `drop_duplicates`; `fillna`; `to_numeric`
- Calls out to: `DraftValueAnalyzer._infer_position_from_slots`, `DraftValueAnalyzer._season_points_from_lineups`, `DraftValueAnalyzer.normalize_player_name`, `copy`, `d.merge`, `draft.copy`, `fillna`, `len`, `map`, `out.drop_duplicates`, `out.merge`, `pd.to_numeric`, `print`
- Called by (static reference): `DraftValueAnalyzer.run_full_pipeline`, `NotebookWorkflow.block_enrich_draft`

`DraftValueAnalyzer._infer_position_from_slots` (lines 571-597)

- Kind: method
- Signature: `DraftValueAnalyzer._infer_position_from_slots(self, lineups) -> pd.DataFrame`
- Intent: Infer player position from lineup slots.
- Validation/Error handling: no explicit raises in this callable; errors are mostly delegated to called APIs.
- Core data operations: `groupby` (Performs grouped aggregations to convert event-level rows into season or week summaries.); `drop_duplicates`; `sort_values`; `reset_index`
- Calls out to: `counts.drop_duplicates`, `isin`, `lineups.copy`, `map`, `rename`, `reset_index`, `size`, `sort_values`, `x.groupby`
- Called by (static reference): `DraftValueAnalyzer.enrich_draft_data`

`DraftValueAnalyzer._season_points_from_lineups` (lines 599-608)

- Kind: method
- Signature: `DraftValueAnalyzer._season_points_from_lineups(self, lineups) -> pd.DataFrame`
- Intent: Compute season total points from lineups.
- Validation/Error handling: no explicit raises in this callable; errors are mostly delegated to called APIs.
- Core data operations: `groupby` (Performs grouped aggregations to convert event-level rows into season or week summaries.); `to_numeric`; `reset_index`
- Calls out to: `df.groupby`, `lineups.copy`, `pd.to_numeric`, `rename`, `reset_index`, `sum`
- Called by (static reference): `DraftValueAnalyzer.enrich_draft_data`

`DraftValueAnalyzer.compute_optimal_startable_points` (lines 612-675)

- Kind: method
- Signature: `DraftValueAnalyzer.compute_optimal_startable_points(self, lineups, slot_counts, flex_eligible, status_every) -> pd.DataFrame`
- Intent: Compute optimal startable points for each team-week.
- Validation/Error handling: contains 1 explicit `raise` statements or guard failures.
- Core data operations: `groupby` (Performs grouped aggregations to convert event-level rows into season or week summaries.); `merge` (Joins datasets to align draft, lineup, transaction, and baseline signals by shared keys.); `fillna`; `astype`; `concat`
- Calls out to: `DraftValueAnalyzer._build_player_week_points`, `DraftValueAnalyzer._choose_optimal_lineup_flex_eligible`, `DraftValueAnalyzer._infer_player_position_by_core_starts`, `ValueError`, `astype`, `enumerate`, `fillna`, `float`, `len`, `lineups.copy`, `map`, `pd.concat`, `print`, `pw.groupby`, `pw.merge`

- Called by (static reference): `DraftValueAnalyzer.run_full_pipeline`, `NotebookWorkflow.block_compute_valid_p`
- Detailed execution notes:
 - This method builds one player-week row per league-year-team-week and chooses an optimal lineup under roster constraints. It is used for two later baselines: valid draft points and waiver valid points.
 - It infers player positions from core slot starts (QB/RB/WR/TE/K/DST), then applies greedy slot filling (core positions first, FLEX second).
 - The output keeps every candidate player row and marks `SelectedOptimal=True` for chosen players. That marker is reused by `add_valid_points` and `compute_valid_waiver_points`.

`DraftValueAnalyzer._normalize_slot_defense` (lines 677-680)

- Kind: method
- Signature: `DraftValueAnalyzer._normalize_slot_defense(self, slot) -> str`
- Intent: Normalize defense slot names.
- Validation/Error handling: no explicit raises in this callable; errors are mostly delegated to called APIs.
- Core data operations: mostly control flow/IO logic, not heavy DataFrame transforms.
- Calls out to: `get`, `str`, `strip`
- Called by (static reference): notebook entrypoint or external caller, or no internal references detected.

`DraftValueAnalyzer._infer_player_position_by_core_starts` (lines 682-704)

- Kind: method
- Signature: `DraftValueAnalyzer._infer_player_position_by_core_starts(self, lineups) -> pd.DataFrame`
- Intent: Infer player position from core slot starts.
- Validation/Error handling: contains 2 explicit `raise` statements or guard failures.
- Core data operations: `groupby` (Performs grouped aggregations to convert event-level rows into season or week summaries.); `agg` (Computes multi-metric summaries (mean/count/std/quantiles) used in baseline construction.); `value_counts`; `reset_index`
- Calls out to: `ValueError`, `agg`, `copy`, `core_rows.groupby`, `isin`, `lineups.copy`, `map`, `rename`, `reset_index`, `s.value_counts`
- Called by (static reference): `DraftValueAnalyzer.compute_optimal_startable_points`, `DraftValueAnalyzer.com`

`DraftValueAnalyzer._build_player_week_points` (lines 706-721)

- Kind: method
- Signature: `DraftValueAnalyzer._build_player_week_points(self, lineups) -> pd.DataFrame`
- Intent: Build player-week points table.
- Validation/Error handling: contains 1 explicit `raise` statements or guard failures.
- Core data operations: `groupby` (Performs grouped aggregations to convert event-level rows into season or week summaries.); `fillna`; `to_numeric`; `reset_index`
- Calls out to: `ValueError`, `df.groupby`, `fillna`, `lineups.copy`, `map`, `max`, `pd.to_numeric`, `rename`, `reset_index`
- Called by (static reference): `DraftValueAnalyzer.compute_optimal_startable_points`

`DraftValueAnalyzer._choose_optimal_lineup_for_group` (lines 723-770)

- Kind: method
- Signature: `DraftValueAnalyzer._choose_optimal_lineup_for_group(self, g, slot_counts, flex_eligible) -> pd.DataFrame`
- Intent: Choose optimal lineup for a single team-week using greedy selection.
- Validation/Error handling: no explicit raises in this callable; errors are mostly delegated to called APIs.

- Core data operations: `sort_values`
- Calls out to: `cand.head`, `g.copy`, `isin`, `select_flex`, `select_top`, `set`, `slot_counts.get`, `sort_values`, `tolist`, `used.update`
- Called by (static reference): `DraftValueAnalyzer.compute_optimal_startable_points`, `DraftValueAnalyzer.compute_optimal_startable_points`
- Detailed execution notes:
 - Implements the greedy selector for one team-week. It tracks a used set so one player cannot fill multiple slots.
 - Selection order matters: required core slots are filled first, then FLEX uses remaining RB/WR/TE players. This approximates lineup optimization quickly without brute-force combinatorics.

`DraftValueAnalyzer.add_valid_points` (lines 772-803)

- Kind: method
- Signature: `DraftValueAnalyzer.add_valid_points(self, draft_enriched, optimal_selected) -> pd.DataFrame`
- Intent: Add season total valid points to draft data.
- Validation/Error handling: contains 2 explicit `raise` statements or guard failures.
- Core data operations: `groupby` (Performs grouped aggregations to convert event-level rows into season or week summaries.); `merge` (Joins datasets to align draft, lineup, transaction, and baseline signals by shared keys.); `fillna`; `to_numeric`; `reset_index`
- Calls out to: `ValueError`, `draft_enriched.merge`, `fillna`, `groupby`, `len`, `pd.to_numeric`, `print`, `rename`, `reset_index`, `sum`
- Called by (static reference): `DraftValueAnalyzer.run_full_pipeline`, `NotebookWorkflow.block_compute_valid_points`

`DraftValueAnalyzer.run_full_pipeline` (lines 807-890)

- Kind: method
- Signature: `DraftValueAnalyzer.run_full_pipeline(self, clean_data, filter_standard, filter_scoring, filter_draft_length, compute_optimal, save_intermediate) -> pd.DataFrame`
- Intent: Run the complete analysis pipeline.
- Validation/Error handling: no explicit raises in this callable; errors are mostly delegated to called APIs.
- Core data operations: `to_csv` (Writes intermediate or final outputs for reuse by later notebook stages.)
- Side effects: CSV write side effect
- Calls out to: `DraftValueAnalyzer._add_draft_length`, `DraftValueAnalyzer.add_valid_points`, `DraftValueAnalyzer.clean_raw_data`, `DraftValueAnalyzer.compute_optimal_startable_points`, `DraftValueAnalyzer.enrich_draft_data`, `DraftValueAnalyzer.filter_draft_length`, `DraftValueAnalyzer.filter_standard_leagues`, `DraftValueAnalyzer.load_multi_season_data`, `draft_enriched.to_csv`, `draft_with_valid.to_csv`, `len`, `print`
- Called by (static reference): notebook entrypoint or external caller, or no internal references detected.
- Detailed execution notes:
 - This is the core draft pipeline used directly in notebook cell 14. It orchestrates all major draft stages in strict order: `clean` -> `load` -> `add draft length` -> `league filters` -> `enrich draft rows` -> `compute optimal weekly lineups` -> `aggregate valid season points`.
 - It persists intermediate datasets into data/preprocessed when `save_intermediate=True`, which is why later notebook cells can reuse precomputed CSV outputs without recomputing all joins.
 - It also stores state on the analyzer instance (`lineups_filt`, `optimal_selected`, `draft_with_valid`), so downstream waiver and start/sit code can use already-filtered and aligned league-year subsets.

`DraftValueAnalyzer.compute_expected_values` (lines 894-932)

- Kind: method
- Signature: `DraftValueAnalyzer.compute_expected_values(self, draft_with_valid, estimator, trim, smooth_window, poly_degree) -> Tuple[pd.DataFrame, pd.DataFrame, pd.DataFrame]`
- Intent: Compute expected values using multiple baselines.

- Validation/Error handling: contains 1 explicit `raise` statements or guard failures.
- Core data operations: mostly control flow/IO logic, not heavy `DataFrame` transforms.
- Calls out to: `DraftValueAnalyzer._expected_by_pick_by_year`, `DraftValueAnalyzer._expected_by_pick_pooled`, `DraftValueAnalyzer._fit_polynomial_baseline`, `ValueError`, `print`
- Called by (static reference): `DraftValueAnalyzer.plot_expected_by_pick`, `DraftValueAnalyzer.score_picks`, `NotebookWorkflow.block_score_draft`
- Detailed execution notes:
 - Builds three baseline families from fully autodrafted teams only: per-year means, pooled denoised estimates, and polynomial smoothing.
 - These baselines are later merged into each draft pick to compute points added and z-scores.

`DraftValueAnalyzer._expected_by_pick_by_year` (lines 934-958)

- Kind: method
- Signature: `DraftValueAnalyzer._expected_by_pick_by_year(self, draft_with_valid) -> pd.DataFrame`
- Intent: Compute expected values per year.
- Validation/Error handling: no explicit raises in this callable; errors are mostly delegated to called APIs.
- Core data operations: `groupby` (Performs grouped aggregations to convert event-level rows into season or week summaries.); `merge` (Joins datasets to align draft, lineup, transaction, and baseline signals by shared keys.); `agg` (Computes multi-metric summaries (mean/count/std/quantiles) used in baseline construction.); `sort_values`; `reset_index`
- Calls out to: `agg`, `copy`, `df.groupby`, `df.merge`, `df_auto.groupby`, `draft_with_valid.copy`, `eq`, `reset_index`, `sort_values`
- Called by (static reference): `DraftValueAnalyzer.compute_expected_values`, `DraftValueAnalyzer.plot_per_season`

`DraftValueAnalyzer._expected_by_pick_pooled_denoised` (lines 960-1028)

- Kind: method
- Signature: `DraftValueAnalyzer._expected_by_pick_pooled_denoised(self, draft_with_valid, estimator, trim, smooth_window) -> pd.DataFrame`
- Intent: Compute pooled denoised expected values across all years.
- Validation/Error handling: contains 2 explicit `raise` statements or guard failures.
- Core data operations: `groupby` (Performs grouped aggregations to convert event-level rows into season or week summaries.); `merge` (Joins datasets to align draft, lineup, transaction, and baseline signals by shared keys.); `agg` (Computes multi-metric summaries (mean/count/std/quantiles) used in baseline construction.); `transform`; `rolling` (Applies local smoothing to reduce pick-level noise in expected-value curves.); `fillna`; `astype`; `to_numeric`
- Calls out to: `ValueError`, `agg`, `astype`, `copy`, `df.groupby`, `df.merge`, `df_auto.groupby`, `draft_with_valid.copy`, `eq`, `fillna`, `float`, `mean`, `np.mean`, `np.nanmedian`, `pd.to_numeric`
- Called by (static reference): `DraftValueAnalyzer.compute_expected_values`
- Detailed execution notes:
 - This is the robust baseline constructor. It supports mean, median, trimmed mean, and winsorized mean to reduce outlier sensitivity.
 - After aggregation by Overall pick, it applies centered rolling smoothing to remove pick-level noise before polynomial fitting.

`DraftValueAnalyzer._fit_polynomial_baseline` (lines 1030-1070)

- Kind: method
- Signature: `DraftValueAnalyzer._fit_polynomial_baseline(self, expected_pooled, x_col, y_col, w_col, degree) -> pd.DataFrame`
- Intent: Fit polynomial baseline to expected curve using weighted least squares.
- Validation/Error handling: contains 1 explicit `raise` statements or guard failures.

- Core data operations: `fillna`; `to_numeric`
- Calls out to: `ValueError`, `expected_pooled.copy`, `fillna`, `len`, `np.isfinite`, `np.ones_like`, `np.poly1d`, `np.polyfit`, `p`, `pd.to_numeric`, `to_numpy`
- Called by (static reference): `DraftValueAnalyzer.compute_expected_values`

`DraftValueAnalyzer.score_picks` (lines 1074-1157)

- Kind: method
- Signature: `DraftValueAnalyzer.score_picks(self, draft_with_valid, expected_by_pick_year, expected_by_pick_pooled, expected_by_pick_poly) -> pd.DataFrame`
- Intent: Score all picks against multiple baselines.
- Validation/Error handling: contains 1 explicit `raise` statements or guard failures.
- Core data operations: `merge` (Joins datasets to align draft, lineup, transaction, and baseline signals by shared keys.); `astype`; `to_numeric`
- Calls out to: `DraftValueAnalyzer.compute_expected_values`, `ValueError`, `astype`, `df.merge`, `draft_scored.merge`, `draft_with_valid.copy`, `expected_by_pick_poly.rename`, `expected_by_pick_year.rename`, `len`, `notna`, `np.where`, `pd.to_numeric`, `print`
- Called by (static reference): `NotebookWorkflow.block_score_draft`
- Detailed execution notes:
 - Merges year-level and pooled baselines back into each pick row and computes points-added metrics for all baseline definitions.
 - Z-score columns normalize points-added by spread, enabling cross-pick comparability.

`DraftValueAnalyzer.plot_team_total_valid_points_distribution` (lines 1161-1213)

- Kind: method
- Signature: `DraftValueAnalyzer.plot_team_total_valid_points_distribution(self, draft_with_valid, fully_autodraft_only, bins, title) -> pd.DataFrame`
- Intent: Plot distribution of team total valid points.
- Validation/Error handling: contains 1 explicit `raise` statements or guard failures.
- Core data operations: `astype`
- Side effects: matplotlib/seaborn plotting side effect
- Calls out to: `DraftValueAnalyzer._compute_team_total_valid_points`, `ValueError`, `astype`, `float`, `len`, `np.mean`, `plt.axvline`, `plt.figure`, `plt.hist`, `plt.legend`, `plt.show`, `plt.tight_layout`, `plt.title`, `plt.xlabel`, `plt.ylabel`
- Called by (static reference): notebook entrypoint or external caller, or no internal references detected.

`DraftValueAnalyzer._compute_team_total_valid_points` (lines 1215-1247)

- Kind: method
- Signature: `DraftValueAnalyzer._compute_team_total_valid_points(self, draft_with_valid, fully_autodraft_only) -> pd.DataFrame`
- Intent: Compute team total valid points.
- Validation/Error handling: contains 1 explicit `raise` statements or guard failures.
- Core data operations: `groupby` (Performs grouped aggregations to convert event-level rows into season or week summaries.); `merge` (Joins datasets to align draft, lineup, transaction, and baseline signals by shared keys.); `agg` (Computes multi-metric summaries (mean/count/std/quantiles) used in baseline construction.); `sort_values`; `reset_index`
- Calls out to: `ValueError`, `agg`, `copy`, `df.groupby`, `df.merge`, `draft_with_valid.copy`, `eq`, `rename`, `reset_index`, `sort_values`, `sum`
- Called by (static reference): `DraftValueAnalyzer.plot_team_total_valid_points_distribution`

`DraftValueAnalyzer.plot_expected_by_pick` (lines 1249-1321)

- Kind: method

- Signature: `DraftValueAnalyzer.plot_expected_by_pick(self, expected_by_pick_pooled, expected_by_pick_poly, show_variance, zoom_first_25) -> None`
- Intent: Plot expected valid points by pick with variance bands.
- Validation/Error handling: contains 1 explicit `raise` statements or guard failures.
- Core data operations: `rolling` (Applies local smoothing to reduce pick-level noise in expected-value curves.)
- Side effects: matplotlib/seaborn plotting side effect
- Calls out to: `DraftValueAnalyzer._expected_distribution`, `DraftValueAnalyzer.compute_expected_values`, `ValueError`, `ax.fill_between`, `ax.grid`, `ax.legend`, `ax.plot`, `ax.set_title`, `ax.set_xlabel`, `ax.set_ylabel`, `ax.text`, `copy`, `dist_25.iterrows`, `fig.tight_layout`, `int`
- Called by (static reference): notebook entrypoint or external caller, or no internal references detected.

`DraftValueAnalyzer._expected_distribution` (lines 1323-1352)

- Kind: method
- Signature: `DraftValueAnalyzer._expected_distribution(self, draft_with_valid) -> pd.DataFrame`
- Intent: Compute expected value distribution with percentiles.
- Validation/Error handling: no explicit raises in this callable; errors are mostly delegated to called APIs.
- Core data operations: `groupby` (Performs grouped aggregations to convert event-level rows into season or week summaries.); `merge` (Joins datasets to align draft, lineup, transaction, and baseline signals by shared keys.); `agg` (Computes multi-metric summaries (mean/count/std/quantiles) used in baseline construction.); `quantile`; `sort_values`; `reset_index`
- Calls out to: `agg`, `copy`, `df.groupby`, `df.merge`, `draft_with_valid.copy`, `eq`, `reset_index`, `s.quantile`, `sort_values`
- Called by (static reference): `DraftValueAnalyzer.plot_expected_by_pick`

`DraftValueAnalyzer.plot_human_advantage_by_year` (lines 1354-1414)

- Kind: method
- Signature: `DraftValueAnalyzer.plot_human_advantage_by_year(self, draft_scored, baseline) -> pd.DataFrame`
- Intent: Plot human draft advantage by year.
- Validation/Error handling: contains 1 explicit `raise` statements or guard failures.
- Core data operations: `groupby` (Performs grouped aggregations to convert event-level rows into season or week summaries.); `agg` (Computes multi-metric summaries (mean/count/std/quantiles) used in baseline construction.); `sort_values`; `reset_index`
- Side effects: matplotlib/seaborn plotting side effect
- Calls out to: `ValueError`, `agg`, `copy`, `int`, `len`, `manual.groupby`, `np.arange`, `plt.axhline`, `plt.bar`, `plt.figure`, `plt.legend`, `plt.show`, `plt.text`, `plt.tight_layout`, `plt.title`
- Called by (static reference): notebook entrypoint or external caller, or no internal references detected.

`DraftValueAnalyzer.plot_human_advantage_by_round` (lines 1416-1462)

- Kind: method
- Signature: `DraftValueAnalyzer.plot_human_advantage_by_round(self, draft_scored, baseline) -> pd.DataFrame`
- Intent: Plot human draft advantage by round.
- Validation/Error handling: contains 1 explicit `raise` statements or guard failures.
- Core data operations: `groupby` (Performs grouped aggregations to convert event-level rows into season or week summaries.); `agg` (Computes multi-metric summaries (mean/count/std/quantiles) used in baseline construction.); `to_numeric`; `sort_values`; `reset_index`
- Side effects: matplotlib/seaborn plotting side effect

- Calls out to: `ValueError`, `agg`, `copy`, `len`, `manual.groupby`, `np.arange`, `pd.to_numeric`, `plt.axhline`, `plt.bar`, `plt.figure`, `plt.legend`, `plt.show`, `plt.tight_layout`, `plt.title`, `plt.xlabel`
- Called by (static reference): notebook entrypoint or external caller, or no internal references detected.

`DraftValueAnalyzer.plot_human_advantage_by_position` (lines 1464-1509)

- Kind: method
- Signature: `DraftValueAnalyzer.plot_human_advantage_by_position(self, draft_scored, baseline) -> pd.DataFrame`
- Intent: Plot human draft advantage by position.
- Validation/Error handling: contains 1 explicit `raise` statements or guard failures.
- Core data operations: `groupby` (Performs grouped aggregations to convert event-level rows into season or week summaries.); `agg` (Computes multi-metric summaries (mean/count/std/quantiles) used in baseline construction.); `sort_values`; `reset_index`
- Side effects: matplotlib/seaborn plotting side effect
- Calls out to: `ValueError`, `agg`, `copy`, `len`, `manual.groupby`, `np.arange`, `plt.axhline`, `plt.bar`, `plt.figure`, `plt.legend`, `plt.show`, `plt.tight_layout`, `plt.title`, `plt.xlabel`, `plt.xticks`
- Called by (static reference): notebook entrypoint or external caller, or no internal references detected.

`DraftValueAnalyzer.plot_per_season_expected_values` (lines 1511-1569)

- Kind: method
- Signature: `DraftValueAnalyzer.plot_per_season_expected_values(self, draft_with_valid, zoom_first_25) -> None`
- Intent: Plot expected values separately for each season.
- Validation/Error handling: contains 1 explicit `raise` statements or guard failures.
- Core data operations: `groupby` (Performs grouped aggregations to convert event-level rows into season or week summaries.)
- Side effects: matplotlib/seaborn plotting side effect
- Calls out to: `DraftValueAnalyzer._expected_by_pick_by_year`, `DraftValueAnalyzer._expected_distribution_by_year`, `ValueError`, `ax.fill_between`, `ax.grid`, `ax.legend`, `ax.plot`, `ax.set_title`, `ax.set_xlabel`, `ax.set_ylabel`, `ax.text`, `copy`, `dist_by_year.groupby`, `expected_by_pick_year.groupby`, `fig.tight_layout`
- Called by (static reference): notebook entrypoint or external caller, or no internal references detected.

`DraftValueAnalyzer._expected_distribution_by_year` (lines 1571-1600)

- Kind: method
- Signature: `DraftValueAnalyzer._expected_distribution_by_year(self, draft_with_valid) -> pd.DataFrame`
- Intent: Compute expected distribution by year.
- Validation/Error handling: no explicit raises in this callable; errors are mostly delegated to called APIs.
- Core data operations: `groupby` (Performs grouped aggregations to convert event-level rows into season or week summaries.); `merge` (Joins datasets to align draft, lineup, transaction, and baseline signals by shared keys.); `agg` (Computes multi-metric summaries (mean/count/std/quantiles) used in baseline construction.); `quantile`; `sort_values`; `reset_index`
- Calls out to: `agg`, `copy`, `df.groupby`, `df.merge`, `draft_with_valid.copy`, `eq`, `reset_index`, `s.quantile`, `sort_values`
- Called by (static reference): `DraftValueAnalyzer.plot_per_season_expected_values`

`DraftValueAnalyzer.build_waiver_add_stints` (lines 1604-1687)

- Kind: method

- Signature: `DraftValueAnalyzer.build_waiver_add_stints(self, transactions) -> pd.DataFrame`
- Intent: Build waiver add stints from transaction data.
- Validation/Error handling: contains 2 explicit `raise` statements or guard failures.
- Core data operations: `groupby` (Performs grouped aggregations to convert event-level rows into season or week summaries.); `merge` (Joins datasets to align draft, lineup, transaction, and baseline signals by shared keys.); `astype`; `to_numeric`; `sort_values`; `reset_index`
- Calls out to: `ValueError`, `adds.merge`, `adds.sort_values`, `astype`, `cand.groupby`, `clip`, `copy`, `int`, `isin`, `isna`, `len`, `min`, `notna`, `np.arange`, `np.where`
- Called by (static reference): `NotebookWorkflow.block_waiver_stints`
- Detailed execution notes:
 - Converts transaction events into stint windows for each add event. A stint starts at add week plus optional offset and ends at drop week minus one (or season end).
 - This conversion is critical because waiver value is evaluated over an interval, not a single transaction row.

`DraftValueAnalyzer.compute_valid_waiver_points` (lines 1689-1759)

- Kind: method
- Signature: `DraftValueAnalyzer.compute_valid_waiver_points(self, waiver_stints, optimal_selected) -> pd.DataFrame`
- Intent: Compute valid waiver points for each stint using optimal lineup selections.
- Validation/Error handling: contains 2 explicit `raise` statements or guard failures.
- Core data operations: `groupby` (Performs grouped aggregations to convert event-level rows into season or week summaries.); `merge` (Joins datasets to align draft, lineup, transaction, and baseline signals by shared keys.); `agg` (Computes multi-metric summaries (mean/count/std/quantiles) used in baseline construction.); `fillna`; `astype`; `to_numeric`; `reset_index`
- Calls out to: `ValueError`, `agg`, `astype`, `copy`, `fillna`, `joined.groupby`, `notna`, `np.where`, `optimal_selected.copy`, `pd.to_numeric`, `reset_index`, `set`, `sorted`, `waiver_stints.merge`
- Called by (static reference): `NotebookWorkflow.block_waiver_valid_points`
- Detailed execution notes:
 - Joins stint windows to optimal lineup selections and counts only points in weeks where the player would be in an optimal lineup.
 - This mirrors the draft valid-points framing so waiver and draft metrics are on comparable footing.

`DraftValueAnalyzer.load_transactions` (lines 1761-1788)

- Kind: method
- Signature: `DraftValueAnalyzer.load_transactions(self, path, year) -> pd.DataFrame`
- Intent: Load and normalize transaction data for a single year.
- Validation/Error handling: contains 1 explicit `raise` statements or guard failures.
- Core data operations: `astype`; `to_numeric`; `read_csv` (Loads persisted raw/intermediate CSV inputs.)
- Calls out to: `DraftValueAnalyzer.normalize_player_name`, `ValueError`, `astype`, `df.copy`, `int`, `map`, `pd.read_csv`, `pd.to_numeric`, `strip`, `upper`
- Called by (static reference): `DraftValueAnalyzer.load_multi_season_transactions`

`DraftValueAnalyzer.load_multi_season_transactions` (lines 1790-1834)

- Kind: method
- Signature: `DraftValueAnalyzer.load_multi_season_transactions(self, lineups_filt) -> pd.DataFrame`
- Intent: Load transaction data across multiple seasons.
- Validation/Error handling: contains 1 explicit `raise` statements or guard failures.
- Core data operations: `merge` (Joins datasets to align draft, lineup, transaction, and baseline signals by shared keys.); `drop_duplicates`; `concat`

- Calls out to: `DraftValueAnalyzer.load_transactions`, `FileNotFoundError`, `drop_duplicates`, `int`, `isdigit`, `iterdir`, `len`, `p.is_dir`, `parts.append`, `pd.concat`, `print`, `sorted`, `tpath.exists`, `transactions_all.merge`
- Called by (static reference): `NotebookWorkflow.block_load_transactions`

`DraftValueAnalyzer.compute_waiver_baseline_candidates` (lines 1836-1890)

- Kind: method
- Signature: `DraftValueAnalyzer.compute_waiver_baseline_candidates(self, waiver_with_valid) -> Tuple[Dict[str, float], pd.DataFrame]`
- Intent: Compute waiver baseline candidates using minimal competency framing.
- Validation/Error handling: contains 2 explicit `raise` statements or guard failures.
- Core data operations: `groupby` (Performs grouped aggregations to convert event-level rows into season or week summaries.); `agg` (Computes multi-metric summaries (mean/count/std/quantiles) used in baseline construction.); `quantile`; `fillna`; `astype`; `to_numeric`; `reset_index`
- Calls out to: `ValueError`, `agg`, `astype`, `copy`, `fillna`, `float`, `mean`, `median`, `pd.to_numeric`, `print`, `quantile`, `reset_index`, `set`, `sorted`, `sum`
- Called by (static reference): `NotebookWorkflow.block_waiver_baselines`
- Detailed execution notes:
 - Creates baseline candidates from stint-week rates and team-season rates. `q25_team_season` is treated as a minimal competency baseline analogous to autodraft for draft value.
 - Returns both the scalar candidates and team-season frame used for exploration plots.

`DraftValueAnalyzer.compute_startsit_metrics` (lines 1892-2014)

- Kind: method
- Signature: `DraftValueAnalyzer.compute_startsit_metrics(self, lineups_filt, slot_counts, flex_eligible) -> Tuple[pd.DataFrame, pd.DataFrame]`
- Intent: Compute Start/Sit metrics using projected-optimal baseline.
- Validation/Error handling: contains 1 explicit `raise` statements or guard failures.
- Core data operations: `groupby` (Performs grouped aggregations to convert event-level rows into season or week summaries.); `merge` (Joins datasets to align draft, lineup, transaction, and baseline signals by shared keys.); `agg` (Computes multi-metric summaries (mean/count/std/quantiles) used in baseline construction.); `fillna`; `astype`; `to_numeric`; `concat`; `reset_index`
- Calls out to: `DraftValueAnalyzer._choose_optimal_lineup_for_group`, `DraftValueAnalyzer._infer_player_pos`, `ValueError`, `actual_wk.merge`, `agg`, `astype`, `copy`, `describe`, `df.groupby`, `eq`, `fillna`, `g.copy`, `get`, `groupby`, `int`
- Called by (static reference): `NotebookWorkflow.block_startsit`
- Detailed execution notes:
 - Computes weekly start/sit points added by comparing actual starters to a projected-optimal lineup (not hindsight-optimal).
 - Applies a completeness filter so only team-weeks with full expected starters in both actual and projected-optimal lineups are scored in the clean output.

`DraftValueAnalyzer.plot_waiver_baseline_exploration` (lines 2018-2075)

- Kind: method
- Signature: `DraftValueAnalyzer.plot_waiver_baseline_exploration(self, team_season, baseline_candidates, waiver_baseline_name) -> None`
- Intent: Plot waiver baseline exploration visuals.
- Validation/Error handling: contains 1 explicit `raise` statements or guard failures.
- Core data operations: `groupby` (Performs grouped aggregations to convert event-level rows into season or week summaries.); `agg` (Computes multi-metric summaries (mean/count/std/quantiles) used in baseline construction.); `quantile`; `astype`; `to_numeric`; `reset_index`
- Side effects: matplotlib/seaborn plotting side effect

- Calls out to: `ValueError`, `agg`, `astype`, `baseline_candidates.get`, `baseline_candidates.items`, `pd.to_numeric`, `plt.axhline`, `plt.axvline`, `plt.figure`, `plt.grid`, `plt.hist`, `plt.legend`, `plt.plot`, `plt.show`, `plt.tight_layout`
- Called by (static reference): `NotebookWorkflow.plot_waiver_baseline_exploration`

`DraftValueAnalyzer.plot_draft_vs_waiver_points` (lines 2077-2183)

- Kind: method
- Signature: `DraftValueAnalyzer.plot_draft_vs_waiver_points(self, draft_scored, waiver_stints, optimal_selected, waiver_baseline_value, draft_points_col, manual_draft_only, season_end_week, ignore_weeks, agg_mode) -> None`
- Intent: Plot draft vs waiver points added over time.
- Validation/Error handling: no explicit raises in this callable; errors are mostly delegated to called APIs.
- Core data operations: `groupby` (Performs grouped aggregations to convert event-level rows into season or week summaries.); `merge` (Joins datasets to align draft, lineup, transaction, and baseline signals by shared keys.); `agg` (Computes multi-metric summaries (mean/count/std/quantiles) used in baseline construction.); `drop_duplicates`; `fillna`; `astype`; `to_numeric`; `value_counts`
- Side effects: matplotlib/seaborn plotting side effect
- Calls out to: `active_stints_week.merge`, `astype`, `copy`, `draft_scored.copy`, `draft_weekly.cumsum`, `drop_duplicates`, `exp_rows.extend`, `fillna`, `float`, `int`, `isin`, `joined.groupby`, `max`, `min`, `notna`
- Called by (static reference): `NotebookWorkflow.plot_cumulative_draft_waiver`

`DraftValueAnalyzer.plot_yearly_draft_waiver_totals` (lines 2185-2260)

- Kind: method
- Signature: `DraftValueAnalyzer.plot_yearly_draft_waiver_totals(self, draft_scored, waiver_with_valid, baseline_candidates, manual_draft_only) -> pd.DataFrame`
- Intent: Plot yearly total points added over expected for Draft + Waiver.
- Validation/Error handling: no explicit raises in this callable; errors are mostly delegated to called APIs.
- Core data operations: `groupby` (Performs grouped aggregations to convert event-level rows into season or week summaries.); `merge` (Joins datasets to align draft, lineup, transaction, and baseline signals by shared keys.); `agg` (Computes multi-metric summaries (mean/count/std/quantiles) used in baseline construction.); `fillna`; `astype`; `to_numeric`; `sort_values`; `reset_index`
- Side effects: matplotlib/seaborn plotting side effect
- Calls out to: `agg`, `astype`, `baseline_candidates.get`, `copy`, `d.groupby`, `d_year.merge`, `draft_scored.copy`, `fillna`, `float`, `len`, `np.arange`, `pd.to_numeric`, `plt.axhline`, `plt.bar`, `plt.figure`
- Called by (static reference): `NotebookWorkflow.plot_yearly_draft_waiver_totals`

`DraftValueAnalyzer.plot_startsit_by_year` (lines 2264-2306)

- Kind: method
- Signature: `DraftValueAnalyzer.plot_startsit_by_year(self, startsit_weekly_clean) -> pd.DataFrame`
- Intent: Plot average Start/Sit points added by year.
- Validation/Error handling: contains 1 explicit `raise` statements or guard failures.
- Core data operations: `groupby` (Performs grouped aggregations to convert event-level rows into season or week summaries.); `agg` (Computes multi-metric summaries (mean/count/std/quantiles) used in baseline construction.); `astype`; `to_numeric`; `sort_values`; `reset_index`
- Side effects: matplotlib/seaborn plotting side effect
- Calls out to: `ValueError`, `agg`, `astype`, `int`, `len`, `np.arange`, `pd.to_numeric`, `plt.axhline`, `plt.bar`, `plt.figure`, `plt.show`, `plt.text`, `plt.tight_layout`, `plt.title`, `plt.xticks`
- Called by (static reference): `NotebookWorkflow.plot_startsit_by_year`

DraftValueAnalyzer.plot_cumulative_draft_waiver_startsit (lines 2308-2424)

- Kind: method
- Signature: `DraftValueAnalyzer.plot_cumulative_draft_waiver_startsit(self, draft_scored, waiver_stints, optimal_selected, startsit_weekly_clean, waiver_baseline_value, draft_points_col, manual_draft_only, season_end_week, ignore_weeks, agg_mode) -> None`
- Intent: Plot cumulative points added: Draft + Waiver + Start/Sit.
- Validation/Error handling: no explicit raises in this callable; errors are mostly delegated to called APIs.
- Core data operations: `groupby` (Performs grouped aggregations to convert event-level rows into season or week summaries.); `merge` (Joins datasets to align draft, lineup, transaction, and baseline signals by shared keys.); `agg` (Computes multi-metric summaries (mean/count/std/quantiles) used in baseline construction.); `drop_duplicates`; `fillna`; `astype`; `to_numeric`; `value_counts`
- Side effects: matplotlib/seaborn plotting side effect
- Calls out to: `active_stints_week.merge`, `astype`, `copy`, `draft_scored.copy`, `draft_weekly.cumsum`, `drop_duplicates`, `exp_rows.extend`, `fillna`, `float`, `int`, `isin`, `joined.groupby`, `max`, `min`, `notna`
- Called by (static reference): `NotebookWorkflow.plot_cumulative_draft_waiver_startsit`
- Detailed execution notes:
 - Integrates all three decision baselines over week time: draft assigned at week 0, waiver by active stint-weeks, and start/sit by weekly totals.
 - Supports `mean_per_team_season` normalization for comparable season-level interpretation.

File: `src\analysis\notebook_workflow.py`

Module role: Notebook-friendly orchestration layer for the fantasy football analysis pipeline.

NotebookWorkflow.__init__ (lines 63-65)

- Kind: method
- Signature: `NotebookWorkflow.__init__(self) -> None`
- Intent: No explicit docstring; behavior inferred from implementation.
- Validation/Error handling: no explicit raises in this callable; errors are mostly delegated to called APIs.
- Core data operations: mostly control flow/IO logic, not heavy DataFrame transforms.
- Calls out to: `DraftValueAnalyzer`, `WorkflowState`
- Called by (static reference): notebook entrypoint or external caller, or no internal references detected.

NotebookWorkflow.block_clean_raw (lines 69-71)

- Kind: method
- Signature: `NotebookWorkflow.block_clean_raw(self) -> None`
- Intent: Notebook block: drop duplicates in raw CSV files.
- Validation/Error handling: no explicit raises in this callable; errors are mostly delegated to called APIs.
- Core data operations: mostly control flow/IO logic, not heavy DataFrame transforms.
- Calls out to: `clean_raw_data`
- Called by (static reference): notebook entrypoint or external caller, or no internal references detected.

NotebookWorkflow.block_load_data (lines 73-78)

- Kind: method
- Signature: `NotebookWorkflow.block_load_data(self) -> Tuple[pd.DataFrame, pd.DataFrame]`
- Intent: Notebook block: load multi-season draft + lineup data.
- Validation/Error handling: no explicit raises in this callable; errors are mostly delegated to called APIs.

- Core data operations: mostly control flow/IO logic, not heavy DataFrame transforms.
- Calls out to: `load_multi_season_data`
- Called by (static reference): `NotebookWorkflow.block_draft_pipeline`, `NotebookWorkflow.block_filter_leagues`

`NotebookWorkflow.block_filter_leagues` (lines 80-120)

- Kind: method
- Signature: `NotebookWorkflow.block_filter_leagues(self) -> Tuple[pd.DataFrame, pd.DataFrame]`
- Intent: Notebook block: apply standard league filters.
- Validation/Error handling: no explicit raises in this callable; errors are mostly delegated to called APIs.
- Core data operations: mostly control flow/IO logic, not heavy DataFrame transforms.
- Calls out to: `NotebookWorkflow.block_load_data`, `_add_draft_length`, `filter_draft_length`, `filter_scoring_rule_outliers`, `filter_standard_leagues`
- Called by (static reference): `NotebookWorkflow.block_draft_pipeline`, `NotebookWorkflow.block_enrich_draft`, `NotebookWorkflow.block_load_transactions`, `NotebookWorkflow.block_startsit`

`NotebookWorkflow.block_enrich_draft` (lines 122-134)

- Kind: method
- Signature: `NotebookWorkflow.block_enrich_draft(self) -> pd.DataFrame`
- Intent: Notebook block: enrich filtered draft data with position + season totals.
- Validation/Error handling: no explicit raises in this callable; errors are mostly delegated to called APIs.
- Core data operations: `to_csv` (Writes intermediate or final outputs for reuse by later notebook stages.)
- Side effects: CSV write side effect
- Calls out to: `NotebookWorkflow.block_filter_leagues`, `draft_enriched.to_csv`, `enrich_draft_data`
- Called by (static reference): `NotebookWorkflow.block_compute_valid_points`, `NotebookWorkflow.block_draft_pi`

`NotebookWorkflow.block_compute_valid_points` (lines 136-155)

- Kind: method
- Signature: `NotebookWorkflow.block_compute_valid_points(self) -> pd.DataFrame`
- Intent: Notebook block: compute optimal lineup selections and add valid points to draft rows.
- Validation/Error handling: no explicit raises in this callable; errors are mostly delegated to called APIs.
- Core data operations: `to_csv` (Writes intermediate or final outputs for reuse by later notebook stages.)
- Side effects: CSV write side effect
- Calls out to: `NotebookWorkflow.block_enrich_draft`, `add_valid_points`, `compute_optimal_startable_points`, `draft_with_valid.to_csv`
- Called by (static reference): `NotebookWorkflow.block_draft_pipeline`, `NotebookWorkflow.block_score_draft`, `NotebookWorkflow.block_waiver_valid_points`

`NotebookWorkflow.block_score_draft` (lines 157-174)

- Kind: method
- Signature: `NotebookWorkflow.block_score_draft(self) -> pd.DataFrame`
- Intent: Notebook block: compute expected curves and score draft picks against baselines.
- Validation/Error handling: no explicit raises in this callable; errors are mostly delegated to called APIs.
- Core data operations: `to_csv` (Writes intermediate or final outputs for reuse by later notebook stages.)
- Side effects: CSV write side effect
- Calls out to: `NotebookWorkflow.block_compute_valid_points`, `compute_expected_values`, `draft_scored.to_csv`, `score_picks`

- Called by (static reference): `NotebookWorkflow.block_draft_pipeline`, `NotebookWorkflow.plot_cumulative_draft`, `NotebookWorkflow.plot_cumulative_draft_waiver_starts`, `NotebookWorkflow.plot_yearly_draft_waiver_t`

`NotebookWorkflow.block_draft_pipeline` (lines 176-186)

- Kind: method
- Signature: `NotebookWorkflow.block_draft_pipeline(self) -> pd.DataFrame`
- Intent: One-line draft pipeline.
- Validation/Error handling: no explicit raises in this callable; errors are mostly delegated to called APIs.
- Core data operations: mostly control flow/IO logic, not heavy `DataFrame` transforms.
- Calls out to: `NotebookWorkflow.block_compute_valid_points`, `NotebookWorkflow.block_enrich_draft`, `NotebookWorkflow.block_filter_leagues`, `NotebookWorkflow.block_load_data`, `NotebookWorkflow.block_score`
- Called by (static reference): notebook entrypoint or external caller, or no internal references detected.
- Detailed execution notes:
 - Wrapper pipeline equivalent to manual draft steps. It executes the same stage order as the notebook but through one method call.

`NotebookWorkflow.block_load_transactions` (lines 190-196)

- Kind: method
- Signature: `NotebookWorkflow.block_load_transactions(self) -> pd.DataFrame`
- Intent: Notebook block: load transactions and align to filtered league-years.
- Validation/Error handling: no explicit raises in this callable; errors are mostly delegated to called APIs.
- Core data operations: mostly control flow/IO logic, not heavy `DataFrame` transforms.
- Calls out to: `NotebookWorkflow.block_filter_leagues`, `load_multi_season_transactions`
- Called by (static reference): `NotebookWorkflow.block_trade_stats`, `NotebookWorkflow.block_waiver_pipeline`, `NotebookWorkflow.block_waiver_stints`

`NotebookWorkflow.block_waiver_stints` (lines 198-217)

- Kind: method
- Signature: `NotebookWorkflow.block_waiver_stints(self) -> pd.DataFrame`
- Intent: Notebook block: build waiver add stints.
- Validation/Error handling: no explicit raises in this callable; errors are mostly delegated to called APIs.
- Core data operations: mostly control flow/IO logic, not heavy `DataFrame` transforms.
- Calls out to: `NotebookWorkflow.block_load_transactions`, `build_waiver_add_stints`
- Called by (static reference): `NotebookWorkflow.block_waiver_pipeline`, `NotebookWorkflow.block_waiver_valid_`

`NotebookWorkflow.block_waiver_valid_points` (lines 219-229)

- Kind: method
- Signature: `NotebookWorkflow.block_waiver_valid_points(self) -> pd.DataFrame`
- Intent: Notebook block: compute valid waiver points by stint.
- Validation/Error handling: no explicit raises in this callable; errors are mostly delegated to called APIs.
- Core data operations: mostly control flow/IO logic, not heavy `DataFrame` transforms.
- Calls out to: `NotebookWorkflow.block_compute_valid_points`, `NotebookWorkflow.block_waiver_stints`, `compute_valid_waiver_points`
- Called by (static reference): `NotebookWorkflow.block_waiver_baselines`, `NotebookWorkflow.block_waiver_pipeline`

`NotebookWorkflow.block_waiver_baselines` (lines 231-238)

- Kind: method

- Signature: `NotebookWorkflow.block_waiver_baselines(self) -> Dict[str, float]`
- Intent: Notebook block: compute waiver baseline candidates.
- Validation/Error handling: no explicit raises in this callable; errors are mostly delegated to called APIs.
- Core data operations: mostly control flow/IO logic, not heavy DataFrame transforms.
- Calls out to: `NotebookWorkflow.block_waiver_valid_points`, `compute_waiver_baseline_candidates`
- Called by (static reference): `NotebookWorkflow.block_waiver_pipeline`, `NotebookWorkflow.plot_waiver_baselines`

`NotebookWorkflow.block_waiver_pipeline` (lines 240-245)

- Kind: method
- Signature: `NotebookWorkflow.block_waiver_pipeline(self) -> Dict[str, float]`
- Intent: One-line waiver pipeline.
- Validation/Error handling: no explicit raises in this callable; errors are mostly delegated to called APIs.
- Core data operations: mostly control flow/IO logic, not heavy DataFrame transforms.
- Calls out to: `NotebookWorkflow.block_load_transactions`, `NotebookWorkflow.block_waiver_baselines`, `NotebookWorkflow.block_waiver_stints`, `NotebookWorkflow.block_waiver_valid_points`
- Called by (static reference): `NotebookWorkflow.plot_cumulative_draft_waiver`, `NotebookWorkflow.plot_cumulative_draft_waiver_totals`, `NotebookWorkflow.plot_yearly_draft_waiver_totals`
- Detailed execution notes:
 - Wrapper chain for transaction load, stint construction, valid point aggregation, and baseline calculation.

`NotebookWorkflow.block_startsit` (lines 249-264)

- Kind: method
- Signature: `NotebookWorkflow.block_startsit(self) -> Tuple[pd.DataFrame, pd.DataFrame]`
- Intent: Notebook block: compute projected-baseline start/sit metrics.
- Validation/Error handling: no explicit raises in this callable; errors are mostly delegated to called APIs.
- Core data operations: `groupby` (Performs grouped aggregations to convert event-level rows into season or week summaries.); `reset_index`
- Calls out to: `NotebookWorkflow.block_filter_leagues`, `compute_startsit_metrics`, `reset_index`, `sum`, `weekly_clean.groupby`
- Called by (static reference): `NotebookWorkflow.plot_cumulative_draft_waiver_startsit`, `NotebookWorkflow.plot_startsit_by_year`
- Detailed execution notes:
 - Wrapper around `compute_startsit_metrics` plus team-season aggregation for plotting convenience.

`NotebookWorkflow.block_trade_stats` (lines 268-289)

- Kind: method
- Signature: `NotebookWorkflow.block_trade_stats(self) -> pd.DataFrame`
- Intent: Notebook block: basic trade action summary by year.
- Validation/Error handling: no explicit raises in this callable; errors are mostly delegated to called APIs.
- Core data operations: `groupby` (Performs grouped aggregations to convert event-level rows into season or week summaries.); `astype`; `sort_values`; `reset_index`
- Calls out to: `NotebookWorkflow.block_load_transactions`, `astype`, `contains`, `copy`, `reset_index`, `size`, `sort_values`, `strip`, `trade_rows.groupby`, `upper`
- Called by (static reference): notebook entrypoint or external caller, or no internal references detected.

NotebookWorkflow.plot_waiver_baseline_exploration (lines 293-300)

- Kind: method
- Signature: NotebookWorkflow.plot_waiver_baseline_exploration(self, waiver_baseline_name) -> None
- Intent: No explicit docstring; behavior inferred from implementation.
- Validation/Error handling: no explicit raises in this callable; errors are mostly delegated to called APIs.
- Core data operations: mostly control flow/IO logic, not heavy DataFrame transforms.
- Calls out to: NotebookWorkflow.block_waiver_baselines, plot_waiver_baseline_exploration
- Called by (static reference): NotebookWorkflow.plot_waiver_baseline_exploration

NotebookWorkflow.plot_cumulative_draft_waiver (lines 302-327)

- Kind: method
- Signature: NotebookWorkflow.plot_cumulative_draft_waiver(self) -> None
- Intent: No explicit docstring; behavior inferred from implementation.
- Validation/Error handling: no explicit raises in this callable; errors are mostly delegated to called APIs.
- Core data operations: agg (Computes multi-metric summaries (mean/count/std/quantiles) used in baseline construction.)
- Calls out to: NotebookWorkflow.block_score_draft, NotebookWorkflow.block_waiver_pipeline, plot_draft_vs_waiver_points
- Called by (static reference): notebook entrypoint or external caller, or no internal references detected.

NotebookWorkflow.plot_yearly_draft_waiver_totals (lines 329-339)

- Kind: method
- Signature: NotebookWorkflow.plot_yearly_draft_waiver_totals(self) -> pd.DataFrame
- Intent: No explicit docstring; behavior inferred from implementation.
- Validation/Error handling: no explicit raises in this callable; errors are mostly delegated to called APIs.
- Core data operations: mostly control flow/IO logic, not heavy DataFrame transforms.
- Calls out to: NotebookWorkflow.block_score_draft, NotebookWorkflow.block_waiver_pipeline, plot_yearly_draft_waiver_totals
- Called by (static reference): NotebookWorkflow.plot_yearly_draft_waiver_totals

NotebookWorkflow.plot_startsit_by_year (lines 341-344)

- Kind: method
- Signature: NotebookWorkflow.plot_startsit_by_year(self) -> pd.DataFrame
- Intent: No explicit docstring; behavior inferred from implementation.
- Validation/Error handling: no explicit raises in this callable; errors are mostly delegated to called APIs.
- Core data operations: mostly control flow/IO logic, not heavy DataFrame transforms.
- Calls out to: NotebookWorkflow.block_startsit, plot_startsit_by_year
- Called by (static reference): NotebookWorkflow.plot_startsit_by_year

NotebookWorkflow.plot_cumulative_draft_waiver_startsit (lines 346-375)

- Kind: method
- Signature: NotebookWorkflow.plot_cumulative_draft_waiver_startsit(self) -> None
- Intent: No explicit docstring; behavior inferred from implementation.
- Validation/Error handling: no explicit raises in this callable; errors are mostly delegated to called APIs.

- Core data operations: `agg` (Computes multi-metric summaries (mean/count/std/quantiles) used in baseline construction.)
- Calls out to: `NotebookWorkflow.block_score_draft`, `NotebookWorkflow.block_startsit`, `NotebookWorkflow.block_waiver_pipeline`, `plot_cumulative_draft_waiver_startsit`
- Called by (static reference): `NotebookWorkflow.plot_cumulative_draft_waiver_startsit`

File: `src\analysis\injury_analyzer.py`

Module role: Injury Impact Analysis for Fantasy Football

`InjuryAnalyzer.__init__` (lines 32-82)

- Kind: method
- Signature: `InjuryAnalyzer.__init__(self, injury_dir, lineup_data_path, years, verbose)`
-> `None`
- Intent: Initialize the `InjuryAnalyzer`.
- Validation/Error handling: contains 1 explicit `raise` statements or guard failures.
- Core data operations: `merge` (Joins datasets to align draft, lineup, transaction, and baseline signals by shared keys.)
- Calls out to: `FileNotFoundError`, `Path`, `path.exists`, `print`, `str`
- Called by (static reference): `notebook` entrypoint or external caller, or no internal references detected.

`InjuryAnalyzer.load_injury_data` (lines 84-176)

- Kind: method
- Signature: `InjuryAnalyzer.load_injury_data(self)` -> `pd.DataFrame`
- Intent: Load injury data from CSV files for all specified years.
- Validation/Error handling: contains 1 explicit `raise` statements or guard failures.
- Core data operations: `concat`; `value_counts`; `read_csv` (Loads persisted raw/intermediate CSV inputs.)
- Calls out to: `DraftValueAnalyzer.normalize_player_name`, `FileNotFoundError`, `all_injuries.append`, `apply`, `contains`, `copy`, `glob`, `injury_file.exists`, `injury_str.lower`, `int`, `isna`, `len`, `list`, `max`, `min`
- Called by (static reference): `InjuryAnalyzer.calculate_player_baselines`, `InjuryAnalyzer.merge_injury_lineup`, `InjuryAnalyzer.run_full_analysis`

`InjuryAnalyzer.load_lineup_data` (lines 178-268)

- Kind: method
- Signature: `InjuryAnalyzer.load_lineup_data(self)` -> `pd.DataFrame`
- Intent: Load lineup data from CSV file.
- Validation/Error handling: contains 2 explicit `raise` statements or guard failures.
- Core data operations: `fillna`; `concat`; `read_csv` (Loads persisted raw/intermediate CSV inputs.)
- Calls out to: `DraftValueAnalyzer.normalize_player_name`, `FileNotFoundError`, `Path`, `ValueError`, `all_lineups.append`, `any`, `apply`, `base_dir.glob`, `fillna`, `int`, `isfile`, `len`, `lineup_file.exists`, `max`, `min`
- Called by (static reference): `InjuryAnalyzer.calculate_player_baselines`, `InjuryAnalyzer.merge_injury_lineup`, `InjuryAnalyzer.run_full_analysis`

`InjuryAnalyzer.calculate_player_baselines` (lines 270-325)

- Kind: method
- Signature: `InjuryAnalyzer.calculate_player_baselines(self)` -> `pd.DataFrame`
- Intent: Calculate each player's baseline performance (healthy weeks only).

- Validation/Error handling: no explicit raises in this callable; errors are mostly delegated to called APIs.
- Core data operations: `groupby` (Performs grouped aggregations to convert event-level rows into season or week summaries.); `merge` (Joins datasets to align draft, lineup, transaction, and baseline signals by shared keys.); `agg` (Computes multi-metric summaries (mean/count/std/quantiles) used in baseline construction.); `fillna`; `reset_index`
- Calls out to: `InjuryAnalyzer.load_injury_data`, `InjuryAnalyzer.load_lineup_data`, `agg`, `copy`, `fillna`, `healthy_weeks.groupby`, `len`, `mean`, `merge`, `print`, `reset_index`, `sum`, `time.time`
- Called by (static reference): `InjuryAnalyzer.merge_injury_lineup_data`, `InjuryAnalyzer.run_full_analysis`
- Detailed execution notes:
 - Defines each players baseline as healthy-week average point differential (actual - projected), then stores variance and sample size metadata.

`InjuryAnalyzer.merge_injury_lineup_data` (lines 327-394)

- Kind: method
- Signature: `InjuryAnalyzer.merge_injury_lineup_data(self) -> pd.DataFrame`
- Intent: Merge injury data with lineup data to create analysis dataset.
- Validation/Error handling: no explicit raises in this callable; errors are mostly delegated to called APIs.
- Core data operations: `merge` (Joins datasets to align draft, lineup, transaction, and baseline signals by shared keys.); `fillna`
- Calls out to: `InjuryAnalyzer.calculate_player_baselines`, `InjuryAnalyzer.load_injury_data`, `InjuryAnalyzer.load_lineup_data`, `fillna`, `len`, `merge`, `merged.merge`, `print`, `sum`
- Called by (static reference): `InjuryAnalyzer.plot_heatmap_position_status`, `InjuryAnalyzer.plot_heatmap_type_status`, `InjuryAnalyzer.plot_heatmap_type_status`, `InjuryAnalyzer.plot_point_differential_distribution`, `InjuryAnalyzer.plot_top_injury_impacts`, `InjuryAnalyzer.prepare_analysis_data`, `InjuryAnalyzer.run_full_analysis`
- Detailed execution notes:
 - Joins injury reports and lineup records on season/week/player normalized name, fills non-matches as No Injury, and attaches baseline columns for downstream deltas.

`InjuryAnalyzer.prepare_analysis_data` (lines 396-435)

- Kind: method
- Signature: `InjuryAnalyzer.prepare_analysis_data(self) -> pd.DataFrame`
- Intent: Prepare analysis dataset with impact metrics. This is a fast step that can be run separately.
- Validation/Error handling: no explicit raises in this callable; errors are mostly delegated to called APIs.
- Core data operations: `merge` (Joins datasets to align draft, lineup, transaction, and baseline signals by shared keys.); `fillna`
- Calls out to: `InjuryAnalyzer.merge_injury_lineup_data`, `copy`, `fillna`, `len`, `print`, `time.time`
- Called by (static reference): `InjuryAnalyzer.analyze_injury_impact`, `InjuryAnalyzer.compute_aggregated_stats`

`InjuryAnalyzer.compute_aggregated_stats` (lines 437-496)

- Kind: method
- Signature: `InjuryAnalyzer.compute_aggregated_stats(self, analysis_df) -> pd.DataFrame`
- Intent: Compute aggregated statistics grouped by injury type, status, and position.
- Validation/Error handling: no explicit raises in this callable; errors are mostly delegated to called APIs.
- Core data operations: `groupby` (Performs grouped aggregations to convert event-level rows into season or week summaries.); `agg` (Computes multi-metric summaries (mean/count/std/quantiles) used in baseline construction.); `fillna`; `reset_index`
- Calls out to: `InjuryAnalyzer.prepare_analysis_data`, `agg`, `copy`, `fillna`, `injured_df.groupby`, `len`, `pd.DataFrame`, `print`, `reset_index`, `time.time`

- Called by (static reference): `InjuryAnalyzer.analyze_injury_impact`
- Detailed execution notes:
 - Builds grouped injury impact statistics by `injury_type` x `position` x `status`, including central tendency, spread, and population counts.

`InjuryAnalyzer.analyze_injury_impact` (lines 498-526)

- Kind: method
- Signature: `InjuryAnalyzer.analyze_injury_impact(self, chunked) -> Tuple[pd.DataFrame, pd.DataFrame]`
- Intent: Perform comprehensive injury impact analysis.
- Validation/Error handling: no explicit raises in this callable; errors are mostly delegated to called APIs.
- Core data operations: `agg` (Computes multi-metric summaries (mean/count/std/quantiles) used in baseline construction.)
- Calls out to: `InjuryAnalyzer.compute_aggregated_stats`, `InjuryAnalyzer.prepare_analysis_data`, `len`, `print`, `time.time`
- Called by (static reference): `InjuryAnalyzer.plot_top_injury_impacts`, `InjuryAnalyzer.run_full_analysis`
- Detailed execution notes:
 - Thin orchestration wrapper that prepares analysis rows and grouped aggregates, returning both granular and summarized outputs.

`InjuryAnalyzer.plot_heatmap_type_position` (lines 528-615)

- Kind: method
- Signature: `InjuryAnalyzer.plot_heatmap_type_position(self, save_path, min_population, show_sample_sizes) -> plt.Figure`
- Intent: Create heatmap: Injury Type (rows) ? Position (columns), color = avg point differential.
- Validation/Error handling: no explicit raises in this callable; errors are mostly delegated to called APIs.
- Core data operations: `groupby` (Performs grouped aggregations to convert event-level rows into season or week summaries.); `merge` (Joins datasets to align draft, lineup, transaction, and baseline signals by shared keys.); `pivot` (Reshapes grouped results into matrix form for signature inference or heatmaps.); `astype`; `sort_values`; `reset_index`
- Side effects: matplotlib/seaborn plotting side effect
- Calls out to: `InjuryAnalyzer.merge_injury_lineup_data`, `astype`, `ax.set_title`, `ax.set_xlabel`, `ax.set_ylabel`, `copy`, `count_pivot.drop`, `count_pivot.reindex`, `df.groupby`, `heatmap_data.merge`, `heatmap_data.pivot`, `heatmap_pivot.copy`, `heatmap_pivot.drop`, `heatmap_pivot.mean`, `heatmap_pivot.reindex`
- Called by (static reference): notebook entrypoint or external caller, or no internal references detected.

`InjuryAnalyzer.plot_heatmap_type_status` (lines 617-703)

- Kind: method
- Signature: `InjuryAnalyzer.plot_heatmap_type_status(self, save_path, min_population, show_sample_sizes) -> plt.Figure`
- Intent: Create heatmap: Injury Type (rows) ? Status (columns), color = avg point differential.
- Validation/Error handling: no explicit raises in this callable; errors are mostly delegated to called APIs.
- Core data operations: `groupby` (Performs grouped aggregations to convert event-level rows into season or week summaries.); `merge` (Joins datasets to align draft, lineup, transaction, and baseline signals by shared keys.); `pivot` (Reshapes grouped results into matrix form for signature inference or heatmaps.); `astype`; `sort_values`; `reset_index`
- Side effects: matplotlib/seaborn plotting side effect
- Calls out to: `InjuryAnalyzer.merge_injury_lineup_data`, `astype`, `ax.set_title`, `ax.set_xlabel`, `ax.set_ylabel`, `copy`, `count_pivot.drop`, `count_pivot.reindex`, `df.groupby`, `heatmap_data.merge`,

- `heatmap_data.pivot, heatmap_pivot.copy, heatmap_pivot.drop, heatmap_pivot.mean, heatmap_pivot.reindex`
- Called by (static reference): notebook entrypoint or external caller, or no internal references detected.

InjuryAnalyzer.plot_heatmap_position_status (lines 705-785)

- Kind: method
- Signature: `InjuryAnalyzer.plot_heatmap_position_status(self, save_path, min_population, show_sample_sizes) -> plt.Figure`
- Intent: Create heatmap: Position (rows) ? Status (columns), color = avg point differential.
- Validation/Error handling: no explicit raises in this callable; errors are mostly delegated to called APIs.
- Core data operations: `groupby` (Performs grouped aggregations to convert event-level rows into season or week summaries.); `merge` (Joins datasets to align draft, lineup, transaction, and baseline signals by shared keys.); `pivot` (Reshapes grouped results into matrix form for signature inference or heatmaps.); `astype`; `sort_values`; `reset_index`
- Side effects: matplotlib/seaborn plotting side effect
- Calls out to: `InjuryAnalyzer.merge_injury_lineup_data`, `astype`, `ax.set_title`, `ax.set_xlabel`, `ax.set_ylabel`, `copy`, `count_pivot.reindex`, `df.groupby`, `heatmap_data.merge`, `heatmap_data.pivot`, `heatmap_pivot.copy`, `heatmap_pivot.mean`, `heatmap_pivot.reindex`, `int`, `mean`
- Called by (static reference): notebook entrypoint or external caller, or no internal references detected.

InjuryAnalyzer.plot_point_differential_distribution (lines 787-915)

- Kind: method
- Signature: `InjuryAnalyzer.plot_point_differential_distribution(self, save_path) -> plt.Figure`
- Intent: Plot distribution of point differentials for injured vs healthy players.
- Validation/Error handling: no explicit raises in this callable; errors are mostly delegated to called APIs.
- Core data operations: `merge` (Joins datasets to align draft, lineup, transaction, and baseline signals by shared keys.); `value_counts`
- Side effects: matplotlib/seaborn plotting side effect
- Calls out to: `InjuryAnalyzer.merge_injury_lineup_data`, `ax1.axvline`, `ax1.axvspan`, `ax1.grid`, `ax1.hist`, `ax1.legend`, `ax1.set_title`, `ax1.set_xlabel`, `ax1.set_ylabel`, `ax2.axvline`, `ax2.grid`, `ax2.hist`, `ax2.legend`, `ax2.set_title`, `ax2.set_xlabel`
- Called by (static reference): notebook entrypoint or external caller, or no internal references detected.

InjuryAnalyzer.run_full_analysis (lines 917-994)

- Kind: method
- Signature: `InjuryAnalyzer.run_full_analysis(self, save_results, output_dir) -> Tuple[pd.DataFrame, pd.DataFrame]`
- Intent: Run the complete injury analysis pipeline.
- Validation/Error handling: no explicit raises in this callable; errors are mostly delegated to called APIs.
- Core data operations: `merge` (Joins datasets to align draft, lineup, transaction, and baseline signals by shared keys.); `agg` (Computes multi-metric summaries (mean/count/std/quantiles) used in baseline construction.); `to_csv` (Writes intermediate or final outputs for reuse by later notebook stages.)
- Side effects: CSV write side effect
- Calls out to: `InjuryAnalyzer.analyze_injury_impact`, `InjuryAnalyzer.calculate_player_baselines`, `InjuryAnalyzer.load_injury_data`, `InjuryAnalyzer.load_lineup_data`, `InjuryAnalyzer.merge_injury_lineup`, `Path`, `agg_stats.to_csv`, `individual_records.to_csv`, `output_dir.mkdir`, `print`, `time.time`
- Called by (static reference): notebook entrypoint or external caller, or no internal references detected.
- Detailed execution notes:

- End-to-end injury pipeline wrapper: load datasets -> compute healthy baselines -> merge -> aggregate analysis -> optional CSV export.

InjuryAnalyzer.plot_top_injury_impacts (lines 996-1086)

- Kind: method
- Signature: `InjuryAnalyzer.plot_top_injury_impacts(self, agg_stats, top_n, save_path)`
-> `plt.Figure`
- Intent: Create bar chart showing top N most impactful injury combinations.
- Validation/Error handling: contains 1 explicit `raise` statements or guard failures.
- Core data operations: `merge` (Joins datasets to align draft, lineup, transaction, and baseline signals by shared keys.); `agg` (Computes multi-metric summaries (mean/count/std/quantiles) used in baseline construction.); `astype`; `sort_values`
- Side effects: matplotlib/seaborn plotting side effect
- Calls out to: `InjuryAnalyzer.analyze_injury_impact`, `InjuryAnalyzer.merge_injury_lineup_data`, `ValueError`, `agg_stats.sort_values`, `agg_stats_sorted.head`, `agg_stats_sorted.tail`, `astype`, `ax1.axvline`, `ax1.barh`, `ax1.grid`, `ax1.set_title`, `ax1.set_xlabel`, `ax1.set_yticklabels`, `ax1.set_yticks`, `ax1.text`
- Called by (static reference): notebook entrypoint or external caller, or no internal references detected.

4) Package Export Files and Wrappers

- `src/__init__.py` is a package marker only.
- `src/data_fetchers/__init__.py` re-exports `FantasyDataCollector` for easier imports.
- `src/analysis/__init__.py` re-exports `DraftValueAnalyzer`, `InjuryAnalyzer`, `NotebookWorkflow`, and `WorkflowState`.
- `NotebookWorkflow` is the main wrapper layer used to convert many multi-step notebook blocks into one-line calls while preserving state in `WorkflowState`.

5) Optional/Reference Paths Not Normally Executed

- Notebook cells 11 and 12 contain commented data-discovery and background collection logic. They are intentionally disabled because target league discovery and pulls are already completed.
- Notebook cells 70 and 72 contain archived prototype sampling code and are not part of the active production analysis path.
- `espn_random_public_league_fetcher_seeded_final_pull.py` has an `if __name__ == "__main__":` block so it can be run as a standalone harvester script.

6) Execution Summary

The project is a notebook-driven analytics pipeline with class-based reusable modules. Runtime-critical state is stored on `DraftValueAnalyzer` and `NotebookWorkflow.state`, enabling staged analysis (draft -> waiver -> start/sit) and then an injury-specific branch. Data collection modules are separate and can be run independently before analysis.